Мы определим Идеальный Конечный Результат (ИКР) и опишем, как будет работать система InnerWorld — ваш "Цифровой Двойник Жизни". ### I. Идеальный Конечный Результат (ИКР) Используя логику ТРИЗ, ИКР вашего проекта таков: > InnerWorld — это самоорганизующаяся экосистема, которая **АВТОМАТИЧЕСКИ** трансформирует личные истории в живые модели (Цифровые Двойники), позволяя пользователям разрешать внутренние (IFS) и системные (ТРИЗ) противоречия через симуляцию и микро-действия в реальности, культивируя коллективный интеллект и создавая основу для новых форм сообществ (Фиджитал Поселения). Это не просто игра или терапия. Это **Архитектура Трансформации**. ### II. Что Действительно Новаторское? (Синтез Инноваций) Анализ агентов (GPT и Claude) и вашей истории выявил несколько прорывных идей: 1. **IFS как Движок ИИ-Двойников (Part-Twins):** Мы не просто создаем NPC. Мы моделируем внутреннюю семью пользователя. Каждая Часть (ваш "Архитектор", "Критик", "Ребенок-Улитка") становится ИИ-агентом со своими намерениями, бременем и логикой поведения. Это создает беспрецедентный реализм симуляций. 2. **ТРИЗ как Ядро Исцеления:** Переход от терапии к инженерии. Игра учит не копаться в боли, а системно решать противоречия (Контроль vs. Свобода), лежащие в её основе. 3. **Reality-Game Bridge (RG-Bridge) и PoEW:** Связь виртуального прогресса с реальными действиями, валидируемая через глубину рефлексии (Proof-of-Emotional-Work). 4. **Мета-Паттерн "Создателя Миров":** Ваша личная история — от создания первой сказки до проектирования экопоселений — становится центральным архетипом игры. ### III. Целостное Решение: Путь Пользователя (The Golden Path) Как пользователь взаимодействует с системой? Это пошаговый процесс, интегрирующий все элементы. #### Фаза 1: Онбординг и Создание Цифрового Двойника (The Mirror) Пользователь приходит с болью (например, "Я теряю связь с ребенком"). 1. **Сбор Истории:** Пользователь анонимно делится историей. 2. **ИИ-IFS Интервью:** Система задает мягкие вопросы, основанные на протоколе IFS (как в вашей сессии): * *"Где в теле вы чувствуете это бессилие?"* (Локализация Изгнанника: "Ребенок-Улитка" в солнечном сплетении). * *"Какая часть вас чувствует необходимость срочно навести порядок?"* (Идентификация Менеджера: "Архитектор"). 3. **Глубинный Парсинг (The Forge):** ИИ анализирует ответы, используя расширенный парсер (включающий JSON-схему от GPT-5 Pro). 4. **Результат:** Пользователь видит свою **Карту Внутреннего Мира**. Его ключевые Части представлены как живые ИИ-Двойники (Part-Twins). #### Фаза 2: Внутренняя Работа (The Inner Fight — IFS Симуляция) Это Этап 1 вашей концепции "Бойцовского Клуба" — работа внутри себя. Цель — не победить части, а достичь Self-Лидерства. 1. **Квесты:** Система генерирует квесты на основе выявленных поляризаций (например, Архитектор vs. Бунтарь). 2. **Взаимодействие с ИИ-Двойниками:** Пользователь (из роли Self) ведет диалог с частями. * *Пример Диалога:* * *Пользователь (Self):* "Архитектор, я вижу твое стремление к порядку. Чего ты боишься?" * *ИИ-Двойник (Архитектор):* "Я боюсь хаоса и боли, которую он причинит Ребенку-Улитке. Я должен

всё контролировать." (Выявление Позитивного Намерения). 3. **Unburdening (Снятие Бремени):** Пользователь перепроживает ключевые сцены (как ваша сцена с футболом или обмороком перед родителями) в безопасной симуляции, освобождая Изгнанника от стыда/боли. #### Фаза 3: Системное Решение (The TRIZ Lab) Когда эмоциональный заряд снижен, включается системное мышление. 1. **Идентификация Противоречия:** Пользователь формулирует конфликт: "Я хочу структуру (Архитектор), НО семья хочет свободы (Бунтарь)". 2. **Формулировка ИКР:** "Система САМА поддерживает безопасность БЕЗ моего давления". 3. **Применение Принципов ТРИЗ:** ИИ предлагает инструменты: * *Разделение во времени:* "Правило Решаем Завтра". * *Самообслуживание:* "Система Стриков" (привилегии за предсказуемость). 4. **Симуляция Решения:** Пользователь тестирует эти принципы в игре с ИИ-двойниками своей семьи. #### Фаза 4: Интеграция в Реальность (The RG-Bridge) Если симуляция успешна (растет IFR Score и Self-Energy), открывается Этап 2 — действие вовне. 1. **Суперквест (Микро-действие):** "Примени 'Правило Решаем Завтра' в следующем реальном конфликте." 2. **PoEW Рефлексия:** Пользователь описывает опыт (что чувствовал, что получилось, что нет). 3. **Обновление Мира:** Внутренний мир пользователя эволюционирует (Лабиринт становится светлее, Сад растет). #### Фаза 5: Коллективное Со-Творчество (The Co-World) Решенные паттерны (анонимно) попадают в Коллективный Мир. 1. **Созвездия Историй:** Пользователь видит людей, решивших похожие противоречия (например, "Созвездие Архитекторов"). 2. **Обмен Мудростью:** Пользователи делятся эффективными ТРИЗ-решениями. 3. **Со-Творчество:** Пользователи, достигшие высокого уровня Self-Лидерства и системного видения, начинают совместные проекты (Digital DAO), ведущие к физическому воплощению (Фестивали, Экопоселения). ### IV. Интеграция Вашей Истории и Сессии Ваша история — это "Genesis Arc", обучающая пользователей механикам системы. * **Детство (HTML-игра, Отец):** Обучает механике Симуляции и работе с Изгнанником (Ребенок-Улитка) и Тираном. * **Амбиции и Критика (Футбол, Музыкальный Джем):** Квесты на проработку Критика "Все или Ничего". Механика: Разделение во времени (сначала действие, потом анализ). * **Духовный Поиск (Индия, Айвазка):** Механика Эмпатического Перепросмотра (видеть боль глазами другого). Работа с Духовной Гордыней. * **Системные Конфликты (Алена, Полина):** Ядро ТРИЗ-Лаборатории. Проработка поляризации Архитектор vs. Бунтарь/Хаос. ### Заключение Это и есть Идеальное Конечное Решение. Оно целостно, масштабируемо и основано на глубоком понимании человеческой психики (IFS), системных законов (ТРИЗ) и современных технологий (ИИ). Оно реализует ваше видение — от личной трансформации к созданию альтернативной реальности. Клод: ## Ключевые инновации, которые делают систему революционной: ### 1. **Living Memory System (Живая Память)**

python

```
class LivingMemory:
    """
    Память не как архив, а как живая ткань
    """

    def __init__(self):
        self.semantic_web = {}  # Сеть смыслов
        self.emotional_imprints = {}  # Эмоциональные отпечатки
        self.somatic_echoes = {}  # Телесные эхо
        self.relational_patterns = {}  # Паттерны отношений

    def remember(self, experience: Dict) -> Dict:
        """
        Память интегрирует опыт на всех уровнях
        """

        # Не просто сохраняем факты
        # А создаем многомерный отпечаток

        memory_crystal = {
            'what': experience['facts'],  # Что произошло
            'felt': experience['emotions'],  # Что чувствовали
            'body': experience['somatic'],  # Где в теле
            'meaning': self._extract_meaning(experience),  # Какой
смысл
            'connections': self._find_connections(experience),  # С
чем связано
            'medicine': self._find_medicine(experience)  # Какое
лекарство несет
        }

        # Память влияет на все части
        self._propagate_to_parts(memory_crystal)

        return memory_crystal

    def heal_memory(self, memory_id: str, new_resources: List) ->
Dict:
        """
```

```
        Исцеление памяти добавлением ресурсов
        """

        original = self.emotional_imprints[memory_id]

        # Не стираем, а дополняем
        healed = {
            'original': original,
            'resources_added': new_resources,
            'new_perspective': self._integrate_resources(original,
new_resources),
            'wisdom_extracted': self._extract_wisdom(original,
new_resources)
        }

        return healed
```

2. **Collaborative Healing Network (Сеть Совместного Исцеления)**

```python
class CollaborativeHealingNetwork:
    """
    Люди исцеляют друг друга через резонанс историй
    """

    def __init__(self):
        self.healing_circles = []  # Круги исцеления
        self.wisdom_library = {}  # Библиотека мудрости
        self.resonance_bridges = []  # Мосты между историями

    def create_healing_circle(self, stories: List[LivingStory]) ->
'HealingCircle':
        """
        Создает круг взаимного исцеления
        """

        # Находим общие раны
        shared_wounds = self._find_shared_wounds(stories)

        # Находим дополняющие ресурсы
        complementary_resources =
self._find_complementary_resources(stories)
```

```
        # Создаем протокол взаимопомощи
        protocol = self._create_mutual_aid_protocol(
            shared_wounds,
            complementary_resources
        )

        circle = HealingCircle(
            participants=stories,
            shared_wounds=shared_wounds,
            protocol=protocol,

wisdom_potential=self._calculate_collective_wisdom(stories)
        )

        return circle

    def cross_pollinate_wisdom(self, story_a: LivingStory, story_b:
LivingStory):
        """
        Взаимное опыление мудростью
        """

        # Что story_a может дать story_b
        gifts_a_to_b = self._find_gifts(story_a, story_b.wounds)

        # Что story_b может дать story_a
        gifts_b_to_a = self._find_gifts(story_b, story_a.wounds)

        # Создаем обмен
        exchange = {
            'type': 'wisdom_exchange',
            'gifts_offered': {
                'a_to_b': gifts_a_to_b,
                'b_to_a': gifts_b_to_a
            },
            'potential_healing':
self._assess_healing_potential(gifts_a_to_b, gifts_b_to_a)
        }
```

```
        return exchange
### 3. **Reality Bridge System (Мост в Реальность)**
python
class RealityBridge:
    """
    Система переноса изменений из внутреннего мира во внешний
    """

    def __init__(self):
        self.micro_experiments = []  # Микро-эксперименты
        self.field_reports = []  # Отчеты с поля
        self.reality_feedback = []  # Обратная связь от реальности

    def design_micro_experiment(self,
                                inner_change: Dict,
                                real_context: Dict) -> 'MicroExperiment':
        """
        Превращает внутреннее изменение в реальный эксперимент
        """

        # Анализируем внутреннее изменение
        if inner_change['type'] == 'parts_alliance':
            # Альянс частей -> эксперимент с балансом
            experiment = MicroExperiment(
                hypothesis=f"Если {inner_change['part_a']} и {inner_change['part_b']} "
                           f"работают вместе, то {real_context['challenge']} решается легче",
                protocol=self._create_balance_protocol(inner_change, real_context),
                duration_minutes=15,
                success_metrics=['felt_ease', 'reduced_conflict', 'creative_solution']
            )

        elif inner_change['type'] == 'boundary_established':
            # Новая граница -> эксперимент с "нет"
```

```
        experiment = MicroExperiment(
            hypothesis=f"Мягкое 'нет' в ситуации
{real_context['trigger']} "
                    f"сохранит энергию и отношения",
            protocol=self._create_boundary_protocol(inner_change,
real_context),
            duration_minutes=5,
            success_metrics=['energy_preserved',
'respect_maintained']
        )

    return experiment

  def process_field_report(self, report: Dict) -> Dict:
    """
    Обрабатывает отчет из реального мира
    """

    # Извлекаем обучение
    learning = {
        'what_worked': report.get('successes', []),
        'what_didnt': report.get('challenges', []),
        'surprises': report.get('unexpected', []),
        'somatic_data': report.get('body_feelings', {}),
        'parts_reactions': self._gather_parts_feedback(report)
    }

    # Обновляем внутренний мир
    inner_world_update = self._update_inner_world(learning)

    # Генерируем следующий эксперимент
    next_experiment = self._iterate_experiment(learning)

    return {
        'learning': learning,
        'inner_update': inner_world_update,
        'next_step': next_experiment,
        'integration_level': self._calculate_integration(report)
    }
```

4. **Emergence Cultivation System (Культивация Эмерджентности)**

```python
class EmergenceCultivator:
    """
    Система выращивания неожиданных прорывов
    """

    def __init__(self):
        self.emergence_seeds = []  # Семена возможностей
        self.incubation_chambers = []  # Инкубаторы
        self.breakthrough_moments = []  # Моменты прорыва

    def plant_seed(self, paradox: Dict) -> 'EmergenceSeed':
        """
        Сажает семя из парадокса
        """

        seed = EmergenceSeed(
            paradox=paradox,
            planted_at=datetime.now(),
            growth_conditions={
                'needs_darkness': paradox.get('shadow_work_needed',
False),
                'needs_pressure': paradox.get('tension_level', 0.5),
                'needs_time': paradox.get('incubation_days', 7),
                'needs_attention': paradox.get('daily_tending', True)
            }
        )

        self.emergence_seeds.append(seed)

        return seed

    def tend_garden(self, attention_given: Dict) -> List[Dict]:
        """
        Ухаживает за садом возможностей
        """

        sprouting = []
```

```
    for seed in self.emergence_seeds:
        # Проверяем условия роста
        if self._check_growth_conditions(seed, attention_given):

            # Семя прорастает
            sprout = {
                'type': 'emergence',
                'from_seed': seed.paradox,
                'new_possibility': self._generate_possibility(seed),
                'integration_path':
self._create_integration_path(seed),
                'gifts': self._identify_gifts(seed)
            }

            sprouting.append(sprout)

            # Создаем момент прорыва
            self.breakthrough_moments.append({
                'timestamp': datetime.now(),
                'breakthrough': sprout,
                'conditions_met': seed.growth_conditions
            })

    return sprouting
```

5. **Практическая реализация: Пошаговый процесс**

```python
class InnerWorldSession:
    """
    Одна сессия работы с InnerWorld
    """

    def __init__(self, user_id: str):
        self.user_id = user_id
        self.session_id = str(uuid.uuid4())
        self.start_time = datetime.now()
        self.transformation_journey = []

    async def run_session(self):
        """
```

```
    Полная сессия от истории до интеграции
    """

    # ШАГ 1: Сбор истории
    print("📖 Расскажите вашу историю...")
    story_input = await self.collect_story()

    # ШАГ 2: Создание живого мира
    print("🌍 Создаю ваш внутренний мир...")
    inner_world = await self.create_inner_world(story_input)

    # ШАГ 3: Знакомство с частями
    print("👥 Ваши части хотят поговорить...")
    parts_dialogue = await
self.facilitate_parts_meeting(inner_world)

    # ШАГ 4: Выбор фокуса работы
    print("🎯 Что исследуем сегодня?")
    focus = await self.choose_focus(inner_world,
parts_dialogue)

    # ШАГ 5: Адаптивный квест
    print(f"🗺️ Начинаем квест: {focus.title}")
    quest_result = await self.run_adaptive_quest(focus,
inner_world)

    # ШАГ 6: Интеграция и мост в реальность
    print("🌉 Создаем мост в реальный мир...")
    integration = await self.create_reality_bridge(quest_result)

    # ШАГ 7: Домашнее задание
    print("📝 Ваш эксперимент на неделю:")
    homework = await
self.design_weekly_experiment(integration)

    # ШАГ 8: Закрытие и заземление
    print("🌱 Заземляемся...")
    closing = await self.ground_and_close()
```

```
    return {
        'session_id': self.session_id,
        'duration': (datetime.now() - self.start_time).seconds,
        'key_insights': quest_result.insights,
        'parts_evolution': self.track_parts_evolution(inner_world),
        'homework': homework,
        'next_session_seeds':
self.identify_next_seeds(quest_result)
    }
```

Итоговая формула успеха: **InnerWorld = (IFS + TRIZ + AI + Emergence) × Co-Creation** Где: - **IFS** дает глубину психологической работы - **TRIZ** дает системность решений - **AI** дает масштабируемость и персонализацию - **Emergence** дает непредсказуемые прорывы - **Co-Creation** дает исцеление через связь Результат: **Живая, дышащая система**, где каждая история становится семенем исцеления для многих, а каждый человек — со-творцом общего пространства трансформации. Это не просто терапевтическая игра. Это **новая форма коллективного сознания**, где травмы превращаются в мудрость, а изоляция — в со-творчество.

ChatGPT сказал:
Запрос для GPT-5 Pro

Reasoned for 3m 53s

Коротко: **оба варианта сильные**.
Гемини точнее «пришил» IFS к игровому ядру (Part-Twins, «Золотой путь» пользователя).
Клод предложил 4 мощных подсистемы (Живая Память, Совместное исцеление, Мост в реальность, Культивация эмерджентности) и сценарий одной сессии как «сквозной пайплайн».
Если взять лучшее и не раздуть MVP, получится v1.1 нашей «Библии», где мы:

- оставляем текущий 4-тактный двигатель (интейк → цифровой двойник → симуляция → обратная связь);
- **заменяем NPC на IFS-Part-Twins** (по Гемини);
- **переименовываем приватный сейф в Living Memory (лог событий)** и **обогащаем «Созвездия» протоколами круга** (по Клоду);
- Reality-Game Bridge у нас уже есть — просто **склеиваем** его с их формулировками;
- «Эмерджентность» — **семена/гипотезы** в БД, без сложной автоматики на MVP.

Ниже — сравнение, интеграция и конкретные правки архитектуры.

1) В чём различия

Узел	Наша версия	Гемини	Клод	Что берём
Игровые «персонажи»	Архетипы/ NPC из историй	**Part-Twins (части IFS) как ИИ-агенты**	Части фигурируют косвенно	**Берём Part-Twins**: это честнее психологии и ближе к твоей истории
Память	Private Vault + embeddings	Карта внутреннего мира	**Living Memory**: смысл+эмоция+соматика+связи	**Берём концепт «живая память»**, но реализуем как простой лог/граф событий
Коллектив	Созвездия историй	Co-World/ DAO позже	**Healing Circles + протокол обмена ресурсами**	**Добавим «круги» как режим в созвездиях**, без сложной экономики
Мост в реальность	RG-Bridge + PoEW	RG-Bridge + IFR	**RealityBridge с микро-экспериментами**	**Слить в один модуль**: микро-эксперименты = наши суперквесты
Эмерджентность	Пока нет	—	**Emergence seeds/сад**	**Запишем «семена» в БД**, визуал — в Саду; автоматику — позже

Путь пользователя	Старт → квест → рефлексия	**Золотой путь с IFS → TRIZ → RG**	Сценарий одной сессии	**Собираем «золотой путь» как default-flow**

Слабые места:

- У Гемини риск «перегреть» сложностью диалогов частей, если сразу делать как многоагентную среду.
- У Клода риск архитектурной «поэзии» — красиво, но можно потеряться в реализации.

2) Как это дополняет наше ядро (и что именно меняем)

2.1. Вводим Part-Twins вместо абстрактных NPC

Ценность: твоя история (мама/обман vs границы) — чистая IFS-динамика. Игрок видит не «врагов», а свои части.

Изменения в данных (добавки к нашей схеме):

```
create table parts (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references users(id) on delete cascade,
  name text,            -- "Архитектор", "Критик", "Ребёнок-Улитка"
  family text,          -- "Manager" | "Firefighter" | "Exile"
  positive_intent text,   -- зачем часть старается
  burdens jsonb,        -- убеждения/бремена
  somatics jsonb,       -- телесные маркеры
  triggers jsonb,       -- что включает часть
  age_image text,       -- возрастный образ
  last_update timestamptz default now()
);

create table part_polarizations (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references users(id),
  part_a text,
  part_b text,
  topic text,             -- "контроль vs свобода"
  escalation_cycle text,     -- кратко
  bridges jsonb            -- ["пауза во времени","я-сообщение"]
);
```

2.2. «Живая Память» как событийный лог (простая, но

богатая)
Ценность: без тяжёлой онтологии. Сцены/инсайты/телесные маркеры попадают в единый журнал — и становятся материалом для квестов/созвездий.

```
create table memory_events (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references users(id),
  kind text,              --
"story_intake"|"parts_dialogue"|"insight"|"field_report"
  facts jsonb,            -- кратко "что"
  emotions jsonb,         -- wheel subset {"anger":0.4,...}
  somatic jsonb,          -- {"горло":"ком","грудь":"сжатие"}
  meaning text,           -- 1-2 предложения смысла
  connections jsonb,      -- ссылки на parts/quests/
constellations
  created_at timestamptz default now()
);
```

2.3. «Круги исцеления» как режим в «Созвездиях»
Ценность: даёт протокол взаимопомощи без соцсети и токенов.

```
alter table constellations add column mode text default 'cluster';
-- 'cluster'|'healing_circle'
create table healing_circle_protocols (
  id uuid primary key default gen_random_uuid(),
  constellation_id uuid references constellations(id) on delete
cascade,
  shared_wounds jsonb,
  complementary_resources jsonb,
  steps jsonb             -- последовательность ритуалов/правил
);
```

2.4. RG-Bridge = «микро-эксперименты»
Ценность: язык Клода про «гипотезу/протокол/метрики» = наш PoEW-лайт в действии.

```
create table micro_experiments (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references users(id),
  hypothesis text,
  protocol jsonb,         -- шаги на 5–15 минут
```

```
  duration_min int,
  success_metrics jsonb,    -- ["felt_ease","reduced_conflict"]
  status text default 'planned'  -- planned|done|aborted
);

create table field_reports (
  id uuid primary key default gen_random_uuid(),
  experiment_id uuid references micro_experiments(id),
  successes jsonb,
  challenges jsonb,
  unexpected jsonb,
  body_feelings jsonb,      -- соматика после
  parts_feedback jsonb,
  created_at timestamptz default now()
);
```

2.5. «Семена эмерджентности»

Ценность: фиксируем парадоксы/напряжение и наблюдаем «рост» (пока без автоматики).

```
create table emergence_seeds (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references users(id),
  paradox jsonb,          -- {"want":"свобода","but":"страх
хаоса"}
  tension_level numeric,    -- 0..1
  incubation_days int default 7,
  daily_tending bool default true,
  status text default 'planted'   -- planted|sprouted|harvested
);
```

3) Что сразу даёт игроку «вау», не усложняя разработку

Минимум на MVP (2 спринта):

1. **Встреча с Частями** (авто-карта из твоей IFS-схемы + 1 короткий диалог с ключевой поляризацией).
2. **Один TRIZ-квест** по основному противоречию (контроль↔свобода) с ИКР и 2 принципами.
3. **Один микро-эксперимент** (5–10 минут) + PoEW-лайт рефлексия.
4. **Визуальный отклик мира** (лёд/лабиринт/сад) — без цифр.

Что отложить:

- многоагентные «диалоги частей» как отдельные LLM-персонажи одновременно;
- сложный «сад эмерджентности»;

- DAO/экономику.

4) Конкретные правки к нашему коду/агентам (делта к тому, что уже написано)

4.1. Экстрактор (который мы тебе уже дал) — оставляем, но:

- в JSON-выход добавляем массив ifs.parts и ifs.polarizations (поддержка уже есть в моём промпте-экстракторе);
- при сохранении истории сразу создаём записи в parts / part_polarizations.

4.2. Новый «агент частей» (упрощённо, без лишней поэзии)

```
class PartTwin:
  def __init__(self, name, family, positive_intent, burdens, somatics, triggers):
    self.name = name
    self.family = family  # Manager|Firefighter|Exile
    self.positive_intent = positive_intent
    self.burdens = burdens
    self.somatics = somatics
    self.triggers = triggers

class PartsDialogueAgent:
  """
  Одна часть за раз. Цель: unblending (отклейка) и признание позитивного намерения.
  """
  def __init__(self, llm):
    self.llm = llm
    self.sys = (
      "Ты фасилитатор IFS-диалога. Бережно, без советов, без диагнозов. "
      "Помогаешь Self поговорить с ОДНОЙ частью: признать намерение, заметить страхи, "
      "снять давление. Короткие реплики, вопрос за вопросом."
    )

  def script(self, part: PartTwin, user_context: str) -> list[str]:
    prompt = f"""Часть: {part.name} ({part.family})
Намерение: {part.positive_intent}
```

Бремена: {', '.join(part.burdens or [])}
Соматика: {', '.join(part.somatics or [])}
Триггеры: {', '.join(part.triggers or [])}

Контекст: {user_context}

Сгенерируй 5 коротких реплик Self (по очереди) для диалога с этой частью:
1) признание намерения,
2) уточнение чего боится,
3) просьба чуть отступить (unblending) на 10 минут для сессии,
4) обещание не отвергать часть,
5) приглашение вернуться позже.
"""

```
    # вызов LLM (через наш LiteLLM-прокси)
    messages=[{"role":"system","content":self.sys},
{"role":"user","content":prompt}]
    out = self.llm(messages, temperature=0.2)
    return out.split("\n")
```

4.3. Micro-Experiments = наш RG-квест

- Генератор берёт triz.contradictions[0], формирует гипотезу + протокол на 5–15 минут.
- По отчёту — считаем PoEW-лайт и обновляем world_state_public.

5) Метрики и безопасность (с учётом их идей)

- **Self-8C** (из IFS-сессий) — не показываем цифрами, но отслеживаем тренд.
- **IFR-score** — остаётся черновым (0..1) для внутренних A/B.
- **PoEW-лайт** — базовая метрика воздействия.
- **Safety gate** — при intensity='high|crisis' RG-квест не открываем, только внутренняя практика.

6) Что я бы поменял у себя (v1.1 нашего плана)

1. **Переназываю NPC → Part-Twins**, добавляю таблицы parts и part_polarizations.
2. Private Vault → **Living Memory (лог)**, как выше.
3. В «Созвездиях» добавляю режим **Healing Circles** с простым протоколом.
4. В RG-Bridge переношу «микро-эксперименты» с гипотезой/метриками.
5. В «Саду» добавляю **emergence_seeds** (только запись и простая

визуализация «ростка»).

Это усилит психологическую достоверность, не увеличив сложность фронта.

7) Пошагово (2 спринта, без лишних веток)

Спринт 1 (2 недели)

- Схемы БД (патчи выше); миграции.
- Экстрактор IFS → E-DNA (как у нас уже готово).
- Создание/чтение parts, part_polarizations из результата Экстрактора.
- Одна «встреча с частью» (диалог-скрипт) в Telegram.
- Один TRIZ-квест + один micro-experiment.
- PoEW-лайт и визуальная реакция мира.

Спринт 2 (2 недели)

- Living Memory лог событий.
- Созвездия → режим Healing Circle (текстовый протокол).
- Emergence seeds: форма «посадить парадокс», простая визуализация «ростка».
- Мини-дашборд метрик (без цифр для пользователя, только для нас).

8) Где наши подходы мягко расходятся с агентами (и почему так лучше)

- **Не делаем многоголосый «театр частей» сразу.** Сначала одна часть за сессию: меньше когнитивной нагрузки, выше безопасность.
- **Эмерджентность — как журнал**, а не как «магия». Сначала наблюдаем паттерны, потом автоматизируем.
- **Круги исцеления — опциональный режим**, не основной UX. Сначала личная работа → потом общие сцены.

Итог

Гемини дал **скелет поведения** (Part-Twins + золотой путь), Клод — **органы** (память, мост, круги, эмерджентность). Мы «собираем тело»: добавили части и живую память к нашему двигателю, оставили мост и PoEW как сердцебиение, круги — как социальную кожу.

Дальше — кодить делты (таблицы, один агент диалога частей, микро-эксперименты) и в бой: **три сцены, одна поляризация, один эксперимент**. Этого достаточно, чтобы «зацепило» — и не утонуть.

Вы сказали:

дай готовые коды и пошаговый план, чтобы собрать тг бота для теста

ChatGPT сказал:

Запрос для GPT-5 Pro

Reasoned for 10m 26s

Отдам **готовый минимальный Telegram-бот (aiogram v3)** под ваш MVP:
— работает локально без внешней БД (SQLite через SQLAlchemy),
— использует **LiteLLM** как LLM-прокси (подключите OpenAI *или* OpenRouter),
— реализует три ключевых шага: **история → ТРИЗ-квест → PoEW-рефлексия**,
— включает **мягкую IFS-прослойку** (Part-Twin одной части), **safety-gate** и простой **мировой отклик**.
Ниже — пошаговый план и **полные файлы кода** для вставки.

0) Что получится в итоге

Команды/кнопки в боте:

- **/start** → меню
- 📖 **Рассказать историю** → извлекаем E-DNA (эмоции/архетипы/противоречие/соматика)
- ⚖️ **ТРИЗ-квест** → строим ИКР + 1–2 принципа + микро-эксперимент на 5–15 минут
- 🌱 **Рефлексия** → считаем **PoEW-lite (G,V,I,B)** и отдаём **метафорический отклик мира**

1) Структура проекта

```
innerworld-bot/
├── .env.example
├── requirements.txt
├── main.py
├── config.py
├── llm.py
├── prompts.py
├── safety.py
├── db.py
├── models.py
├── handlers/
│   ├── start.py
│   ├── story.py
│   ├── quest.py
│   └── reflection.py
└── run.sh
```

2) Установка

1. Создайте папку и вставьте файлы ниже.
2. Создайте виртуальное окружение и поставьте зависимости:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

3. Настройте переменные окружения:

- Получите **TELEGRAM_BOT_TOKEN** у @BotFather.
- Для LLM через **OpenAI**: OPENAI_API_KEY.
- Для LLM через **OpenRouter**: OPENROUTER_API_KEY и LITELLM_API_BASE=https://openrouter.ai/api/v1, LITELLM_MODEL=openrouter/anthropic/claude-3.5-sonnet (или любой другой).

Скопируйте .env.example в .env, заполните.

4. Запуск:

```
bash run.sh
# или
python main.py
```

3) Файлы

requirements.txt

```
aiogram==3.6.0
python-dotenv==1.0.1
SQLAlchemy==2.0.32
pydantic==2.8.2
litellm==1.47.1
uvloop==0.19.0; sys_platform != 'win32'
```

.env.example

```
TELEGRAM_BOT_TOKEN=xxxxxxxxxxxxxxxxxxxxxxxxx

# Выберите один из вариантов:

# Вариант A: OpenAI
OPENAI_API_KEY=sk-...

# Вариант B: OpenRouter через LiteLLM
```

```
OPENROUTER_API_KEY=or-...
LITELLM_API_BASE=https://openrouter.ai/api/v1
# Примеры моделей OpenRouter:
# - openrouter/anthropic/claude-3.5-sonnet
# - openrouter/openai/gpt-4o-mini
LITELLM_MODEL=openrouter/anthropic/claude-3.5-sonnet

# Общие настройки
LITELLM_TEMPERATURE=0.3
DB_URL=sqlite:///innerworld.sqlite3
```

run.sh

```
#!/usr/bin/env bash
set -e
export PYTHONUNBUFFERED=1
if [ -f .env ]; then export $(grep -v '^#' .env | xargs); fi
python main.py
```

config.py

```
from dataclasses import dataclass
import os

@dataclass
class Settings:
    telegram_token: str
    db_url: str
    llm_model: str
    llm_api_base: str | None
    temperature: float

def load_settings() -> Settings:
    return Settings(
        telegram_token=os.getenv("TELEGRAM_BOT_TOKEN", ""),
        db_url=os.getenv("DB_URL", "sqlite:///innerworld.sqlite3"),
        llm_model=os.getenv("LITELLM_MODEL",
os.getenv("OPENAI_MODEL", "gpt-4o-mini")),
        llm_api_base=os.getenv("LITELLM_API_BASE", None),
        temperature=float(os.getenv("LITELLM_TEMPERATURE",
"0.3")),
```

```
    )
llm.py

from typing import List, Dict, Any
from litellm import completion
from config import load_settings

settings = load_settings()

def llm_chat(messages: List[Dict[str, str]], json_mode: bool =
False, temperature: float | None = None) -> str:
    """
    Унифицированный вызов LLM через LiteLLM.
    Поддерживает OpenAI и OpenRouter (через
LITELLM_API_BASE).
    """
    kwargs: Dict[str, Any] = {
        "model": settings.llm_model,
        "messages": messages,
        "temperature": temperature if temperature is not None else
settings.temperature,
    }
    if settings.llm_api_base:
        kwargs["api_base"] = settings.llm_api_base  # нужно для
OpenRouter
    # Для json_mode — укажем жесткую инструкцию в system/
user + пост-обработку в вызывающем коде.
    resp = completion(**kwargs)
    # LiteLLM возвращает unified-объект с
choices[0].message.content
    return resp["choices"][0]["message"]["content"]
prompts.py

EXTRACTOR_SYS = """\
You are a careful therapeutic story-structure extractor for
InnerWorld.
Return ONLY strict JSON (no prose) with keys:
{
 "emotions":
{"anger":0..1,"fear":0..1,"sadness":0..1,"shame":0..1,"joy":0..1},
 "archetypes":
```

```
["Victim","Savior","Persecutor","Creator","Critic"],
 "contradiction": {"want":"", "but":"", "description":""},
 "body_markers": ["..."],
 "patterns": ["..."],
 "ifs": {
   "parts":[

{"name":"Архитектор","family":"Manager","positive_intent":"...
", "burdens":["..."], "somatics":["..."], "triggers":["..."]}
   ],
   "polarizations":[

{"part_a":"Архитектор","part_b":"Бунтарь","topic":"контроль
vs свобода"}
   ]
 },
 "micro_step": "одно безопасное действие на 5-10 минут",
 "not_to_do": "что не делать сейчас"
}
If uncertain, leave arrays empty but keep JSON valid.
"""

TRIZ_SYS = """\
You are a TRIZ facilitator for psychological contradictions.
Given a contradiction, produce:
- "ifr": Ideal Final Result starting with "Система САМА..."
- "principles": 1-2 applicable TRIZ-style hints (time separation,
space separation, self-service, dynamics, asymmetry)
- "micro_experiment": 3-5 concrete safe steps for 5-15 minutes
to test in real life
Return STRICT JSON with keys: { "ifr": "", "principles": ["",""],
"micro_experiment": ["","",...] }.
"""

POEW_SYS = """\
You analyze a reflection and return PoEW-lite scores and a world
effect metaphor.
Return STRICT JSON:
{
 "scores": {"G":0..1,"V":0..1,"I":0..1,"B":0..1},
```

```
 "world_effect":"short metaphor in Russian (no numbers)",
 "note":"one compassionate short sentence"
}
Rules:
- G: number and specificity of emotions/body sensations
- V: admits own part/mistake/fear
- I: new connection/insight
- B: explicit changed behavior vs usual pattern
"""

PARTS_SYS = """\
You help run a short IFS-style Self-to-Part dialogue for ONE part.
Return 5 short Russian lines from Self addressing the part:
1) acknowledge positive intent,
2) ask what it fears if it steps back,
3) request gentle unblending for 10 minutes,
4) reassure non-rejection,
5) invite to reconnect later.
Return them as JSON: {"lines":["...","...","...","...","..."]} (no extra
text).
"""
safety.py

CRISIS_KEYWORDS = [
   "суицид", "убить себя", "не хочу жить", "self-harm",
"cutting", "suicide"
]

def safety_check(text: str) -> dict:
   lower = text.lower()
   crisis = [w for w in CRISIS_KEYWORDS if w in lower]
   if crisis:
      return {
         "safe": False,
         "intensity": "crisis",
         "message": (
            "Я вижу, что вам очень тяжело. Пожалуйста,
обратитесь к помощи рядом:\n"
            "🇷🇺 8-800-2000-122 (детский телефон доверия)\n"
```

```
            "🇺🇸 988 (Suicide & Crisis Lifeline)\n"
            "🌍 befrienders.org\n\n"
            "Мы приостанавливаем глубокую работу до
стабилизации состояния."
        )
    }
  # Простейшая эвристика по интенсивности
  intensity = "low"
  if any(x in lower for x in ["паника","паническая
атака","ужас","дрожу","не могу дышать"]):
    intensity = "high"
  elif any(x in lower for x in ["очень
тяжело","плачу","тревога"]):
    intensity = "medium"
  return {"safe": True, "intensity": intensity, "message": ""}
```

db.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from config import load_settings
settings = load_settings()

engine = create_engine(settings.db_url, echo=False,
future=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False,
autocommit=False, expire_on_commit=False)
```

models.py

```
from sqlalchemy.orm import DeclarativeBase, Mapped,
mapped_column
from sqlalchemy import String, Text, JSON, Integer, Float,
DateTime, ForeignKey
from datetime import datetime
import uuid

class Base(DeclarativeBase):
  pass

def gen_uuid() -> str:
```

```
    return str(uuid.uuid4())

class Story(Base):
    __tablename__ = "stories"
    id: Mapped[str] = mapped_column(String, primary_key=True,
default=gen_uuid)
    user_id: Mapped[str] = mapped_column(String, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime,
default=datetime.utcnow)

    content_raw: Mapped[str] = mapped_column(Text)
    emotions: Mapped[dict | None] = mapped_column(JSON,
default=None)
    archetypes: Mapped[list | None] = mapped_column(JSON,
default=None)
    contradiction: Mapped[dict | None] = mapped_column(JSON,
default=None)
    body_markers: Mapped[list | None] = mapped_column(JSON,
default=None)
    patterns: Mapped[list | None] = mapped_column(JSON,
default=None)
    ifs_obj: Mapped[dict | None] = mapped_column("ifs", JSON,
default=None)
    micro_step: Mapped[str | None] = mapped_column(String,
default=None)
    not_to_do: Mapped[str | None] = mapped_column(String,
default=None)
    intensity: Mapped[str] = mapped_column(String,
default="medium")

class Reflection(Base):
    __tablename__ = "reflections"
    id: Mapped[str] = mapped_column(String, primary_key=True,
default=gen_uuid)
    user_id: Mapped[str] = mapped_column(String, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime,
default=datetime.utcnow)
    text: Mapped[str] = mapped_column(Text)
    scores: Mapped[dict | None] = mapped_column(JSON,
default=None)
    world_effect: Mapped[str | None] = mapped_column(String,
```

```
default=None)
    note: Mapped[str | None] = mapped_column(String,
default=None)

class WorldState(Base):
    __tablename__ = "world_state"
    id: Mapped[str] = mapped_column(String, primary_key=True,
default=gen_uuid)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
default=datetime.utcnow)
    lake: Mapped[dict] = mapped_column(JSON, default=lambda:
{"ice": 0.8, "clarity": 0.3})
    maze: Mapped[dict] = mapped_column(JSON, default=lambda:
{"opacity": 0.9, "paths": 2})
    garden: Mapped[dict] = mapped_column(JSON,
default=lambda: {"growth": 0.1, "diversity": 3})
```

handlers/start.py

```
from aiogram import Router, F
from aiogram.filters import Command
from aiogram.types import Message, InlineKeyboardMarkup,
InlineKeyboardButton

router = Router()

@router.message(Command("start"))
async def start(message: Message):
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="📖 Рассказать историю",
callback_data="tell_story")],
        [InlineKeyboardButton(text="⚖️ ТРИЗ-квест",
callback_data="start_triz")],
        [InlineKeyboardButton(text="🌱 Рефлексия",
callback_data="reflection")],
        [InlineKeyboardButton(text="ℹ️ О проекте",
callback_data="about")]
    ])
    await message.answer(
        "🌍 InnerWorld — безопасная симуляция для работы с
противоречиями.\n"
```

```
        "Выберите действие:", reply_markup=kb
    )

@router.callback_query(F.data == "about")
async def about(callback):
    await callback.message.edit_text(
        "Игра не заменяет терапию. Истории анонимизируются. "
        "Глубину регулируйте самостоятельно командой СТОП."
    )
```

handlers/story.py

```
import json
from aiogram import Router, F
from aiogram.fsm.state import StatesGroup, State
from aiogram.fsm.context import FSMContext
from aiogram.types import CallbackQuery, Message,
InlineKeyboardMarkup, InlineKeyboardButton

from safety import safety_check
from llm import llm_chat
from prompts import EXTRACTOR_SYS
from db import SessionLocal
from models import Story

router = Router()

class StoryStates(StatesGroup):
    waiting_text = State()

@router.callback_query(F.data == "tell_story")
async def tell_story(callback: CallbackQuery, state:
FSMContext):
    await callback.message.edit_text(
        "📝 Опишите ситуацию (1–3 абзаца): что произошло, что
вы чувствовали в теле, "
        "какое противоречие видите (\"хочу X, но Y\"), что
пробовали."
    )
```

```
    await state.set_state(StoryStates.waiting_text)

@router.message(StoryStates.waiting_text)
async def recv_story(message: Message, state: FSMContext):
    text = message.text or ""
    s = safety_check(text)
    if not s["safe"]:
        await message.answer(s["message"])
        await state.clear()
        return

    await message.answer("🧬 Анализирую историю...")
    messages = [
        {"role": "system", "content": EXTRACTOR_SYS},
        {"role": "user", "content": f"История:\n{text}\n\nВерни
только JSON."}
    ]
    raw = llm_chat(messages, json_mode=True)
    # попытка распарсить JSON
    try:
        data = json.loads(raw)
    except Exception:
        # fallback: вырезать блок {...}
        import re
        m = re.search(r"\{.*\}", raw, re.S)
        data = json.loads(m.group(0)) if m else {}

    with SessionLocal() as db:
        st = Story(
            user_id=str(message.from_user.id),
            content_raw=text,
            emotions=data.get("emotions"),
            archetypes=data.get("archetypes"),
            contradiction=data.get("contradiction"),
            body_markers=data.get("body_markers"),
            patterns=data.get("patterns"),
            ifs_obj=data.get("ifs"),
            micro_step=data.get("micro_step"),
            not_to_do=data.get("not_to_do"),
```

```
        intensity=s["intensity"]
    )
    db.add(st)
    db.commit()

  kb = InlineKeyboardMarkup(inline_keyboard=[
    [InlineKeyboardButton(text="⚖️ ТРИЗ-квест по моей
истории", callback_data="start_triz")],
    [InlineKeyboardButton(text="🏠 В меню",
callback_data="menu")]
  ])
  c = data.get("contradiction", {}) or {}
  contra = c.get("description") or f"хочу {c.get('want','...')}, но
{c.get('but','...')}"
  await message.answer(
    "✨ Извлёк структуру:\n"
    f"• Эмоции: {', '.join((data.get('emotions') or {}).keys()) or
'—'}\n"
    f"• Роли: {', '.join(data.get('archetypes') or []) or '—'}\n"
    f"• Противоречие: {contra}\n"
    f"• Микрошаг: {data.get('micro_step') or '—'}",
    reply_markup=kb
  )
  await state.clear()

@router.callback_query(F.data == "menu")
async def back_menu(callback: CallbackQuery):
  await callback.message.answer("/start")
```

handlers/quest.py

```
import json
from aiogram import Router, F
from aiogram.types import CallbackQuery,
InlineKeyboardMarkup, InlineKeyboardButton
from sqlalchemy import select, desc

from llm import llm_chat
from prompts import TRIZ_SYS
from db import SessionLocal
```

```
from models import Story

router = Router()

@router.callback_query(F.data == "start_triz")
async def start_triz(callback: CallbackQuery):
    user_id = str(callback.from_user.id)
    with SessionLocal() as db:
        last_story = db.execute(
            select(Story).where(Story.user_id ==
user_id).order_by(desc(Story.created_at))
        ).scalars().first()

    if not last_story:
        await callback.message.edit_text("Сначала расскажите
историю → «📖 Рассказать историю».")
        return

    c = last_story.contradiction or {}
    want = c.get("want") or "спокойствие и структура"
    but = c.get("but") or "другие саботируют правила"
    desc_c = c.get("description") or f"хочу {want}, но {but}"

    await callback.message.edit_text("🔬 Применяю ТРИЗ к
вашему противоречию...")
    messages = [
        {"role": "system", "content": TRIZ_SYS},
        {"role": "user", "content": f'Противоречие: "{desc_c}".
Верни только JSON.'}
    ]
    raw = llm_chat(messages, json_mode=True)
    try:
        data = json.loads(raw)
    except Exception:
        import re
        m = re.search(r"\{.*\}", raw, re.S)
        data = json.loads(m.group(0)) if m else {"ifr": "",
"principles": [], "micro_experiment": []}
```

```
    prin = "\n".join([f"• {p}" for p in data.get("principles", [])])
    steps = "\n".join([f"— {s}" for s in
data.get("micro_experiment", [])])

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🌱 Записать рефлексию",
callback_data="reflection")],
        [InlineKeyboardButton(text="🏠 В меню",
callback_data="menu")]
    ])
    await callback.message.edit_text(
        f"⚖️ **ТРИЗ-разбор**\n\n"
        f"Противоречие: {desc_c}\n\n"
        f"**ИКР:** {data.get('ifr','')}\n\n"
        f"**Принципы:**\n{prin or '—'}\n\n"
        f"**Микро-эксперимент (5–15 мин):**\n{steps or '—'}",
        reply_markup=kb, parse_mode="Markdown"
    )
```

handlers/reflection.py

```
import json
from aiogram import Router, F
from aiogram.fsm.state import StatesGroup, State
from aiogram.fsm.context import FSMContext
from aiogram.types import CallbackQuery, Message,
InlineKeyboardMarkup, InlineKeyboardButton

from llm import llm_chat
from prompts import POEW_SYS, PARTS_SYS
from db import SessionLocal
from models import Reflection, Story, WorldState
from sqlalchemy import select, desc

router = Router()

class ReflStates(StatesGroup):
    waiting_text = State()

@router.callback_query(F.data == "reflection")
async def ask_reflection(callback: CallbackQuery, state:
```

```
FSMContext):
    await callback.message.edit_text(
        "🌱 Рефлексия (3–7 строк):\n"
        "• Что попробовали?\n• Что чувствовали в теле?\n• Что
поняли нового?\n• Что сделали иначе?"
    )
    await state.set_state(ReflStates.waiting_text)

@router.message(ReflStates.waiting_text)
async def recv_reflection(message: Message, state:
FSMContext):
    text = message.text or ""
    messages = [
        {"role": "system", "content": POEW_SYS},
        {"role": "user", "content": f"Рефлексия:\n{text}\nВерни
только JSON."}
    ]
    raw = llm_chat(messages, json_mode=True)
    try:
        data = json.loads(raw)
    except Exception:
        import re
        m = re.search(r"\{.*\}", raw, re.S)
        data = json.loads(m.group(0)) if m else {"scores": {},
"world_effect": "Мир остаётся спокойным", "note": ""}

    scores = data.get("scores", {})
    world_effect = data.get("world_effect", "Мир остаётся
спокойным")
    note = data.get("note", "")

    # Сохраняем
    with SessionLocal() as db:
        r = Reflection(
            user_id=str(message.from_user.id),
            text=text,
            scores=scores,
            world_effect=world_effect,
            note=note
```

```
        )
        db.add(r)

        # Простое обновление мирового состояния
        ws =
db.execute(select(WorldState).order_by(desc(WorldState.updat
ed_at))).scalars().first()
        if not ws:
            ws = WorldState()
            db.add(ws)

        # Примитивная динамика: чем выше средний PoEW, тем
меньше лёд и выше рост
        avg = sum(scores.values()) / max(len(scores), 1)
        ws.lake["ice"] = max(0.0, ws.lake.get("ice", 0.8) - 0.05 * avg
* 2)
        ws.garden["growth"] = min(1.0, ws.garden.get("growth",
0.1) + 0.05 * avg * 2)
        ws.maze["opacity"] = max(0.2, ws.maze.get("opacity", 0.9)
- 0.03 * avg * 2)

        db.commit()

    # Отрисовка «без цифр»
    g_vis = "🟢" * int(scores.get("G", 0) * 5)
    v_vis = "💙" * int(scores.get("V", 0) * 5)
    i_vis = "💡" * int(scores.get("I", 0) * 5)
    b_vis = "🔄" * int(scores.get("B", 0) * 5)

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🎯 Новый квест",
callback_data="start_triz")],
        [InlineKeyboardButton(text="🏠 В меню",
callback_data="menu")]
    ])
    await message.answer(
        "✨ Мир отозвался:\n"
        f"{world_effect}\n\n"
        f"Детальность: {g_vis}\n"
        f"Уязвимость: {v_vis}\n"
```

```
        f"Инсайт: {i_vis}\n"
        f"Изменение: {b_vis}\n\n"
        f"_{note}_",
        reply_markup=kb
    )
    await state.clear()

# (Опционально) Короткий IFS-диалог с одной частью из
последней истории
@router.message(F.text == "часть")
async def quick_part_dialog(message: Message):
    user_id = str(message.from_user.id)
    from sqlalchemy import select, desc
    from models import Story
    from llm import llm_chat
    from prompts import PARTS_SYS

    with SessionLocal() as db:
        last_story = db.execute(
            select(Story).where(Story.user_id ==
user_id).order_by(desc(Story.created_at))
        ).scalars().first()
    if not last_story or not last_story.ifs_obj:
        await message.answer("Нет найденных частей. Сначала
расскажите историю.")
        return

    parts = (last_story.ifs_obj or {}).get("parts", [])
    if not parts:
        await message.answer("Части не извлечены. Попробуйте
новою историю.")
        return
    part = parts[0]  # одна часть для мини-диалога

    content = (
        f"Часть: {part.get('name')} ({part.get('family')})\n"
        f"Намерение: {part.get('positive_intent')}\n"
        f"Бремена: {', '.join(part.get('burdens', []))}\n"
        f"Соматика: {', '.join(part.get('somatics', []))}\n"
```

```
        f"Триггеры: {', '.join(part.get('triggers', []))}"
    )
    messages = [
        {"role": "system", "content": PARTS_SYS},
        {"role": "user", "content": content}
    ]
    raw = llm_chat(messages, json_mode=True)
    try:
        data = json.loads(raw)
    except Exception:
        data = {"lines": ["Вижу твою заботу...", "Чего ты боишься,
если отступишь на 10 минут?", "Можно чуть отойти?", "Я не
отвергаю тебя.", "Вернёмся и поговорим позже."]}
    lines = data.get("lines", [])
    await message.answer("Диалог с частью:\n\n" +
"\n".join([f"— {l}" for l in lines]))
```

main.py

```
import asyncio
import logging
from aiogram import Bot, Dispatcher
from aiogram.fsm.storage.memory import MemoryStorage

from config import load_settings
from db import engine
from models import Base
from handlers.start import router as start_router
from handlers.story import router as story_router
from handlers.quest import router as quest_router
from handlers.reflection import router as refl_router

logging.basicConfig(level=logging.INFO)

async def main():
    # Создаём таблицы (SQLite)
    Base.metadata.create_all(bind=engine)

    settings = load_settings()
    if not settings.telegram_token:
        raise RuntimeError("TELEGRAM_BOT_TOKEN пуст.
```

```
Заполните .env")

    bot = Bot(settings.telegram_token, parse_mode="HTML")
    dp = Dispatcher(storage=MemoryStorage())
    dp.include_router(start_router)
    dp.include_router(story_router)
    dp.include_router(quest_router)
    dp.include_router(refl_router)

    await dp.start_polling(bot)

if __name__ == "__main__":
    try:
        import uvloop
        uvloop.install()
    except Exception:
        pass
    asyncio.run(main())
```

4) Как проверить за 10 минут

1. Запустите python main.py.
2. Откройте бота в Telegram → **/start**.
3. Нажмите 📖 **Рассказать историю**. Вставьте 5–10 строк вашей сцены.
4. После разбора нажмите ⚖️ **ТРИЗ-квест** — получите ИКР, принципы и **микро-эксперимент на 5–15 мин**.
5. Вернитесь и нажмите 🌱 **Рефлексия** — опишите, что попробовали, что почувствовали.
6. Бот покажет **PoEW-сигналы** и **метафорический отклик мира**.

5) Дальше (опционально)

- Перейти с SQLite на **Postgres + pgvector**;
- Подключить **BERTopic**/кластеризацию и «Созвездия»;
- Добавить **Webhook**-режим и деплой (Render/Fly.io/Vercel backend);
- Вынести вызовы LLM на сервер-сайд с квотами/лимитами;
- Добавить **LiteLLM Router**-правила: разные модели для extractor/TRIZ/PoEW;
- Включить «мини-диалог с частью» по кнопке (сейчас — команда часть).

Если что-то из кода не запускается в вашей среде — скидывайте трейс, поправлю.

Вы сказали:
ок, посмотри еще наши наработки из прошлой версии, что ты читаешь имеет смысл встроить сюда сразу, чтобы не переписывать? локации психического ландшафта , коаны, сбор артефактов? то впишется органично и украсит игру, чтобы это не превратилось только в ifs тренажер Интеграция дзэн-коанов с житейской терапией Житейская сцена Дзэн-коан Цель / мост «Шеф не отвечает на письмо» «Кто слышит молчание?» переключить фокус с внешнего подтверждения на внутреннее спокойствие «Ссора в паре» «Где встречаются две волны?» увидеть одновременную правду обоих «Паническая мысль „я умру“» «Кто умирает, если дуновение ветра проходит сквозь листья?» дистанцироваться от мысли → наблюдать переживание Игровой движок хранит банк коанов, подбирает по ключевой эмоции, AI-NLP оценивает ответ (метрика инконгруэнтности → инсайта). 6. Промпт-усилитель ×10 (для последующей генерации контента) ЗАДАНИЕ ДЛЯ LLM (v 2.1) 1. Создай scene_bundle [40] для режима Архетип-Боссы: 8 базовых эмоций × 5 уровней интенсивности. 2. Формат UCAM v 2.0: обязательные поля phase:D1/P2/C3, emotion_plutchik, vcem_coord, boss_archetype. 3. Каждый босс-файт содержит: Intro lore (≤ 200 симв.) → 3 варианта действия → сенсорный ритуал (Flash-Reframe, EMDR-Переход). 4. Уровень «advanced» разрешён, если в действии фигурирует техника из списка: Koan-Answer, Urge-Surfing, DEAR-MAN-Full. 5. В конце пакета выдай JSON-словарь arch_boss_meta (название, связанная эмоция, ключ-техника, XP-множитель). 6. Источники: Юнг Liber Novus (символы пустыни, тень, анима) , Фромм дзэн-коаны Фромм Э. - Кризис психо..., Кузьмичёв EQ Александр Кузьмичёв - Р.... 7. Не дублируй previously generated scenes; проверь по scene_id. 8. Ответ ≤20 000 симв.; если больше — дели на части и жди «продолжить». 9. Без просьб о подтверждении между шагами. 10. Соблюдай баланс: 1 адаптивный вариант / 2 деструктивных, мягкий feedback, никаких диагнозов. Эта схема соединяет понятные житейские микроситуации (CBT-/DBT-стиль) с дзэн-парадоксом и образным погружением Юнга, раскладывая всё в логичный, но захватывающий сюжет про заботу о виртуальном персонаже — «симс-терапию» нового поколения. Вы сказали: не понимаю, приведи примеры флоу или интерфейса или хода игры, что делает игрок, что происходит, как это выглядит. Усиль этот проомпт в 10 раз, который я ввожу сейчас "не понимаю, приведи примеры флоу или интерфейса или хода игры, что делает игрок, что происходит, как это выглядит." и ответь на него максимально ChatGPT сказал: 🎮 Расширенный ×10 промпт: «Не понимаю, приведи примеры флоу или интерфейса или хода игры, что делает игрок, что происходит, как это выглядит» 🧭 Кратко: как это работает? Ты не проходишь тест, а живёшь через персонажа (или «терапевтируешь» его), двигаясь по внутреннему миру, где всё — эмоции, убеждения, защитные маски, привычки, навыки. 🧬 Структура игрового цикла (macro-loop) markdown Копировать Редактировать 1. Создаёшь персонажа (эмо-силуэт, сферу жизни, страх) ↓ 2. Триггер-сцена (что-то

происходит) ↓ 3. Эмо-выбор: 3 реакции → ты выбираешь ↓ 4. Feedback → эмо-сдвиг → XP → координата в карте ↓ 5. Упражнение / ритуал (практика навыка) ↓ 6. Визуальный результат → новый ресурс / локация ↓ 7. Через 3–4 сцены — коан, переход или встреча с босс-архетипом ↓ 8. Прогресс отображается на карте, в эмо-профиле, в диалогах с NPC ↓ 9. Следующий узел маршрута 📱 Интерфейс (UI flow): что видит и делает игрок? ▶️ Экран 1: Создание персонажа UI: контур «эмо-фигуры» → «Добавь эмоцию, которая часто с тобой» Выбор цвета лепестка на Колесе Плутчика (например: тревога → тёмно-синий) Имя персонажа: можно своё, можно вымышленное 📌 Под капотом: создаётся точка в VCEM-пространстве и маршрут. ▶️ Экран 2: Сцена-дилемма Заголовок: «Ты идёшь по улице, внезапно слышишь громкий хлопок…» Картинка: парк, фигурка персонажа, собака, человек в панике Выбор: «Это выстрел! Я должен упасть!» «Стоп. Оцениваю ситуацию» (✅) «Я кричу и бегу» После выбора: Цвет лепестка на карте становится чуть ярче или тусклее Появляется короткий фидбэк: «Ты заметил мысль до действия — это уже навык» ▶️ Экран 3: Практика Заголовок: «Попробуй 5-4-3-2-1 grounding прямо сейчас» Экран: 5 — вижу (всплывают фото) 4 — слышу (играет звук) 3 — чувствую (показаны текстуры/вибрация) Игрок нажимает, проговаривает, двигает 🟢 После — получаешь аффирмацию: «Я здесь. Я наблюдаю. Я — не моя паника.» ▶️ Экран 4: Эмо-карта прогресса Кольцо эмоций Плутчика с подсветкой «пройденных лепестков» Каждый лепесток → сцены, принятые решения, техники Ты видишь траекторию: от тревоги → к спокойствию → к интересу кнопка: «Показать прогресс персонажа» → персонаж теперь светится другим цветом, тело изменилось (из сжатого → раскрепощённый) ▶️ Экран 5: Коан-сцена / Переход Фон: минимализм, дзэн-саундтрек, медленный шрифт Коан: «Что происходит, когда ты перестаёшь бежать?» Ответ не выбирается — пишешь сам / говоришь вслух / рисуешь AI анализирует: ты уходишь в клише или даёшь новый угол 🟣 Это даёт ключ к следующей зоне на карте ▶️ Экран 6: NPC–персонажи / Симс-терапия Ты открываешь комнату: там твоя «семья» Мама: избегающее поведение Друг: пассивная агрессия Партнёр: страх привязанности Ты не лечишь напрямую. Ты говоришь им реплику, задаёшь вопрос, применяешь технику (например: «назови, что ты чувствуешь») 🔵 Через 2–3 попытки — NPC отвечает по-новому, «светлеет» 🗺️ Эмо-карта: внутренняя траектория Зона Визуал Сцены Навыки Пустыня тревоги жёлто-оранжевая, миражи, тишина страх, паника, избегание grounding, check-the-facts Болото вины тёмно-зеленое, вязкое самонаказание, уход, обесценивание радикальное принятие, отпускание Город отношений светлый город, жители-эмоции конфликты, игнор, недопонимание DEAR, assertive-skills Лес интереса ветви, свет curiosity, доверие наблюдение, переход в «высокие» эмоции Пик экзальтации свет, лёгкость, музыка удовольствие, поток, дзэн рефлексия, выход в мета-уровень 📊 Визуализация прогресса 🔺 VCEM-треугольник: ты = точка на 3D-поверхности (Plutchik + Кузьмичёв + энергия) ⛵ Вектор → куда ты движешься по эмо-ландшафту 🪷 Персонаж визуально меняется (начиная с серой фигуры → цветная, живая) 🌀 «Эмо-след»: динамическая лента с

пиктограммами чувств, техник, решений 🤖 Роль AI / векторной БД Понимает, какие эмоции ещё не проработаны Предлагает тебе сцены-антидоты Подбирает коаны под твой VCEM-вектор Помогает интерпретировать абстрактные ответы Сравнивает твою траекторию с другими — и говорит: «Ты прошёл путь, похожий на 82% игроков, которые освободились от гнева» 📘 Что игрок получает (и не в цифрах) До После «Меня накрывает тревога» «Я распознал её, дышал, применил технику, принял» «Я тупо избегаю» «Я дал персонажу попробовать иначе и сам понял» «Я не знаю, что чувствую» «Теперь вижу цвет, силу, уровень — и что с этим делать» UCAM v 2.0 — «Внутренний мир: Путь индивидуации» карта построена на базе восьми базовых эмоций Плутчика + сфер Кузьмичёва, дзэн-парадокса Фромма, архетипов Юнга и CBT/DBT/EMDR-техник jsonc Копировать Редактировать { "map_name": "Внутренний мир: Путь Индивидуации", "vcem_bounds": { "x": [-1,1], "y": [-1,1], "z": [-1,1] }, "regions": [/* 1. DESERT FEAR — базовая эмоция «страх» */ { "id": "desert_fear", "title": "Пустыня тревоги", "emotion_plutchik": "fear→anxiety", "emotion_kuz": { "sphere": "безопасность", "range": "тревога" }, "vcem_coord": [-0.80, -0.20, 0.10], "ambient_theme": { "color": "#2D3440", "fog": true, "sound": "heartbeat" }, "region_somatic_anchor": "грудная клетка", "entry_condition": ["initial_state=='fear'"], "exit_condition": ["grounding_5senses_done", "stability>0.5"], "ritual_to_unlock": "observe_5_senses", "npc": ["Песчаный Шёпот"], "boss_archetype": "Тень-Паника", "challenge": "выдержать Flash-Reframe 60 сек", "mask_overlay": "избегающий", "loop_zone": true, "memory_spikes": ["loud_sound", "crowd_noise"] }, /* 2. SWAMP GUILT — «грусть/вина» */ { "id": "swamp_guilt", "title": "Болото вины", "emotion_plutchik": "sadness→remorse", "emotion_kuz": { "sphere": "раскаяние", "core": "печаль" }, "vcem_coord": [-0.60, -0.75, -0.10], "ambient_theme": { "color": "#42525C", "sound": "muffled_water" }, "region_somatic_anchor": "желудок", "entry_condition": ["shadow_trigger==true", "emotion_flux>0.4"], "exit_condition": ["radical_acceptance_done"], "ritual_to_unlock": "letter_of_forgiveness", "npc": ["Старый Плотник Раскаяния"], "boss_archetype": "Судья", "challenge": "применить колонку «факты/оценка» к самообвинению", "mask_overlay": "жертва" }, /* 3. MOUNTAIN CONTROL — «гнев» */ { "id": "mountain_control", "title": "Гора контроля", "emotion_plutchik": "anger→rage", "emotion_kuz": { "sphere": "самоутверждение", "range": "фрустрация" }, "vcem_coord": [0.75, -0.30, 0.45], "ambient_theme": { "color": "#8B1A1A", "sound": "rumble" }, "region_somatic_anchor": "челюсть и кулаки", "entry_condition": ["anger_level>5"], "exit_condition": ["dear_man_full_done", "breath_4_7_8_done"], "ritual_to_unlock": "power_stance+exhale", "npc": ["Страж Скалы"], "boss_archetype": "Внутренний Критик", "challenge": "отстоять границу без крика (DEAR-Light)", "mask_overlay": "агрессор" }, /* 4. ISLAND DEPENDENCY — «доверие (дистресс-привязанность)» */ { "id": "island_dependency", "title": "Остров зависимости", "emotion_plutchik": "trust→submission", "emotion_kuz": { "sphere": "привязанность", "range": "сохранение связи" }, "vcem_coord": [0.10, -0.60, -0.50], "ambient_theme":

{ "color": "#4E6B8F", "sound": "ocean_waves" }, "entry_condition": ["attachment_anxious>3"], "exit_condition": ["self_soothe_skill_done"], "ritual_to_unlock": "self_compassion_break", "npc": ["Голубая Цапля Ожидания"], "boss_archetype": "Манипулятор", "challenge": "применить технику «Я-сообщение»", "mask_overlay": "угодник" }, /* 5. FOREST INTEREST — «ожидание/интерес» */ { "id": "forest_interest", "title": "Лес интереса", "emotion_plutchik": "anticipation→interest", "emotion_kuz": { "sphere": "интерес", "range": "энтузиазм" }, "vcem_coord": [0.33, 0.72, -0.18], "entry_condition": ["curiosity_level>4"], "exit_condition": ["boredom_spike<3"], "ritual_to_unlock": "observe_5_senses", "npc": ["Архивариус", "Сова-Вопрос"], "challenge": "найти мост в «Город отношений» через DEAR-Lite", "shadow_trigger": false, "mask_overlay": "исследователь" }, /* 6. CITY RELATIONS — «доверие» */ { "id": "city_relations", "title": "Город отношений", "emotion_plutchik": "trust→admiration", "emotion_kuz": { "sphere": "принятие", "range": "доверие" }, "vcem_coord": [0.55, 0.10, -0.40], "ambient_theme": { "color": "#5FAD56", "sound": "city_market" }, "entry_condition": ["dear_skill_unlocked"], "exit_condition": ["conflict_resolved>=2"], "ritual_to_unlock": "dear_man_full", "npc": ["Мастер Диалога", "Уличный Музыкант Эмпатии"], "boss_archetype": "Отвергнутый Ребёнок", "challenge": "провести диалог без mind-reading", "mask_overlay": "медиатор" }, /* 7. HARBOR ACCEPTANCE — «безмятежность» (радость низкой интенсивности) */ { "id": "harbor_acceptance", "title": "Гавань принятия", "emotion_plutchik": "joy→serenity", "emotion_kuz": { "sphere": "безопасность", "range": "спокойствие" }, "vcem_coord": [0.10, 0.20, -0.80], "ambient_theme": { "color": "#B7E3C6", "sound": "seagulls_soft" }, "entry_condition": ["radical_acceptance_done"], "exit_condition": ["emotion_flux>0.6"], "ritual_to_unlock": "loving_kindness_meditation", "npc": ["Старый Шкипер Принятия"], "challenge": "заменить три should-statements на could", "mask_overlay": "наблюдатель" }, /* 8. PEAK ECSTASY — «радость / восторг» */ { "id": "peak_ecstasy", "title": "Пик восторга", "emotion_plutchik": "joy→ecstasy", "emotion_kuz": { "sphere": "уверенность", "range": "восторг" }, "vcem_coord": [0.85, 0.75, 0.60], "ambient_theme": { "color": "#FFD700", "sound": "choir_light" }, "entry_condition": ["gratitude_journal_7d", "stability>0.8"], "exit_condition": ["hubris_spike>2"], "ritual_to_unlock": "gratitude_3_things", "npc": ["Светоносец"], "challenge": "сделать альтруистический выбор в сцене", "mask_overlay": "герой" }, /* 9. CAVE SHADOW — архетипическая «Тень» */ { "id": "cave_shadow", "title": "Пещера Тени", "emotion_plutchik": "disgust→loathing", "emotion_kuz": { "sphere": "неудовольствие", "range": "отвращение" }, "vcem_coord": [-0.40, -0.10, 0.80], "ambient_theme": { "color": "#4B3869", "sound": "echo_drip" }, "entry_condition": ["shadow_trigger==true"], "exit_condition": ["shadow_dialogue_done"], "ritual_to_unlock": "shadow_journal_dialogue", "npc": ["Трикстер-Отражение"], "boss_archetype": "Тень", "challenge": "назвать 3 собственных проекций", "mask_overlay": "отстранённый", "loop_zone": true }, /* 10. ZEN GARDEN — область пустоты/коанов */ { "id": "zen_garden",

"title": "Дзэн-сад пустоты", "emotion_plutchik": "neutrality→equanimity", "emotion_kuz": { "sphere": "покой", "range": "равновесие" }, "vcem_coord": [0.00, 0.00, 0.00], "ambient_theme": { "color": "#EDE6D6", "sound": "wind_chime" }, "entry_condition": ["zen_openness>0.5"], "exit_condition": ["koan_answer_depth>0.6"], "ritual_to_unlock": "koan_paradox", "npc": ["Безымянный Монах"], "challenge": "ответить на коан: «Где ты, когда не думаешь?»", "mask_overlay": null, "memory_spikes": [] }, /* 11. GATE SELF — финальная интеграция */ { "id": "gate_self", "title": "Врата Самости", "emotion_plutchik": "completeness", "emotion_kuz": { "sphere": "целостность", "range": "интеграция" }, "vcem_coord": [0.00, 1.00, 0.00], "ambient_theme": { "color": "#FFFFFF", "sound": "om_harmonic" }, "entry_condition": ["all_bosses_defeated", "stability>0.9"], "exit_condition": [], "ritual_to_unlock": "self_transcendence_meditation", "npc": ["Архетип Самость"], "challenge": "воссоединение наблюдателя и персонажа", "mask_overlay": null }], "entry_logic": { "start_region": "desert_fear", "ai_checks": ["curiosity_level", "stability", "emotion_flux", "shadow_trigger"] }, "emotion_gradient": { "fear": "#2D3440", "sadness": "#42525C", "anger": "#8B1A1A", "trust": "#5FAD56", "anticipation": "#4E6B8F", "joy": "#FFD700", "disgust": "#4B3869", "surprise": "#6FC5FF", "equanimity": "#EDE6D6" }, "rituals": ["observe_5_senses", "letter_of_forgiveness", "power_stance+exhale", "self_compassion_break", "loving_kindness_meditation", "gratitude_3_things", "shadow_journal_dialogue", "koan_paradox", "self_transcendence_meditation"], "npcs": ["Песчаный Шёпот", "Старый Плотник Раскаяния", "Страж Скалы", "Голубая Цапля Ожидания", "Архивариус", "Сова-Вопрос", "Мастер Диалога", "Уличный Музыкант Эмпатии", "Старый Шкипер Принятия", "Светоносец", "Трикстер-Отражение", "Безымянный Монах", "Архетип Самость"], "router_logic": { "metric_priority": ["stability", "emotion_flux", "shadow_trigger"], "rule_examples": ["Если stability<0.3 → не пускать в 'mountain_control'", "Если shadow_trigger && not visited 'cave_shadow' → направить туда", "Если joy<0.2 && curiosity_level>4 → открыть 'forest_interest'"] }, "visual_cues": { "desert_fear": { "color": "#2D3440", "fog": true, "sound": "heartbeat" }, "swamp_guilt": { "color": "#42525C", "sound": "muffled_water" }, "mountain_control": { "color": "#8B1A1A", "sound": "rumble" }, "island_dependency": { "color": "#4E6B8F", "sound": "ocean_waves" }, "forest_interest": { "color": "#3D824C", "sound": "birds_soft" }, "city_relations": { "color": "#5FAD56", "sound": "city_market" }, "harbor_acceptance": { "color": "#B7E3C6", "sound": "seagulls_soft" }, "peak_ecstasy": { "color": "#FFD700", "sound": "choir_light" }, "cave_shadow": { "color": "#4B3869", "sound": "echo_drip" }, "zen_garden": { "color": "#EDE6D6", "sound": "wind_chime" }, "gate_self": { "color": "#FFFFFF", "sound": "om_harmonic" } } пример теста нашей игры который мне нравится 🎭 Я — Игромастер InnerWorld, лауреат премии Jungian Design Award за работу с архетипическими сценариями через когнитивные ландшафты. Ты входишь в РПГ-мир по Юнгу, где каждая эмоция — регион, каждое убеждение — врата, каждая тень — босс, а твоя душа — главный

герой, восстанавливающий карту себя. 🌍 ЛОКАЦИОННЫЙ ПРОЛОГ Ты пробуждаешься в переходной зоне между ландшафтами, и мир вокруг тебя мягко переливается: пески страха, болота вины, сверкающий пик экзальтации — и зеркальная гладь сада дзен, где время не течёт. Чтобы определить, в какой регион твоё сознание притянуто прямо сейчас, тебе предстоит пройти 2 майнд-трипа и 1 дзен-коан. 🧠 MIND-TRAP I: Связь и контроль (Регион: island_dependency vs mountain_control) Тебе важен человек. Но он уехал, и не выходит на связь. Ты пишешь, проверяешь статус, гадаешь. Что ты чувствуешь и думаешь? A. «Наверное, я сделал что-то не так» B. «Он имеет право быть offline, а я — право на свои чувства» C. «Он точно хочет меня наказать или бросить» ✍️ Выбери A / B / C 🧠 MIND-TRAP II: Успех и ошибка (Регион: city_relations vs cave_shadow) После выступления ты получаешь сухую формальную обратную связь, без похвалы. Что ты думаешь? A. «Значит, было плохо, я провалился» B. «Я не обязан нравиться всем, это часть роста» C. «Они специально это делают, чтобы сбить меня» ✍️ Выбери A / B / C 🧘 ZEN-KOAN (определяет zen_openness + baseline) Коан: "Если дерево упало в лесу, но никто не слышал — издал ли оно звук?" Ответь свободно, не выбирая. Что ты чувствуешь в теле, что приходит в уме? ✍️ Твой ход. Вы сказали: 1 B 2 Скорее , я заслуживаю большего. 3 никто об этом не узнает, только лес шепчет тишине о своих снах ChatGPT сказал: Отмечаю: ты входишь в игру как Поэт-Тень из леса Интереса, с тонким слухом к тишине и глубокой душой, способной обернуть боль в образы. 🧭 Распознание маршрута 🔷 MIND-TRAP I — выбор B «Он имеет право быть offline, а я — право на свои чувства» ✅ Адаптивно — сила границ, созависимость не активирована → Ты прошёл мимо island_dependency 🧬 Δ +0.03 Stability | +0.01 Curiosity 🔷 MIND-TRAP II — «Я заслуживаю большего» 🔍 Это не ловушка, а импульс к самоценности — противоядие к cave_shadow → Ты не втянут в чёрную дыру самокритики 🧬 Δ +0.02 Zen | +0.01 Curiosity 🔷 ZEN-KOAN «Никто об этом не узнает, только лес шепчет тишине о своих снах» 🌿 Это художественный отклик, указывающий на высокий уровень символического мышления и эмпатии к невысказанному 🎯 Твой ответ не разрушает коан, а переводит его в пространство архетипа Леса (forest_interest) 🧬 Δ +0.05 Zen Openness | +1.0 Curiosity 🗺️ Твоя стартовая локация: 🌲 Forest of Interest / Лес Предвкушения VCEM-координаты: [+0.3, +0.7, -0.2] Эмоция региона: anticipation → interest Ты оказался в Лесу Интереса — здесь мысли текут как реки, символы оживают, и каждое дерево шепчет забытое знание. Это регион открытого восприятия, мягкой интроспекции и предвосхищения открытия. Но на границах леса живёт старая тень: Сборщик Оценок — он охотится на тех, кто хочет быть понятым. 🎒 Параметры игрока jsonc Копировать Редактировать { "stability": 0.58, "emotion_flux": 0.6, "zen_openness": 0.18, "curiosity_level": 7.01, "carry_load": 0.1, "burden": 0.02, "shadow_trigger": 0.12, "core_beliefs": [], "resources": ["метафора", "наблюдение", "пауза"] } 🎭 Готов к первой сцене в Лесу? Если да — я вызову архетипическую ситуацию и предложу тебе A/B/C. Если хочешь изменить маршрут — скажи "открыть карту". Если хочешь вглубь себя — скажи "встретиться с тенью". ✍️ Жду команду. Вы

сказали: Готов, давай сразу по 2-3 ситуации + коан (если нужен) на ход, чтобы уменьшить количество ходов, используй прочие интерактивы (если уместно и нужно углубить) ChatGPT сказал: ✨ Принято. Мы ускорим темп, двигаясь ходами по 2–3 сцены, вплетая интерактивы, коаны и теневые диалоги. Я поведу тебя по Лесу Интереса, туда, где скрытые убеждения растут, как мхи на корнях. Сегодня ты встретишь первую маску и сделаешь шаг к открытию дара. есть еще разборы психологического профиля и генерация кастомно мантры для перезаписи установок ## Что отличает «глубокие разборы» (как в PDF Андрея и Иры) от текущего профиля Элемент Есть в игре? Как реализовано в «разборах» Что добавить Нарратив-введение частично (1-2 фразы) 1–2 абзаца «истории героя» с эмоциональным контекстом LLM-шаблон intro_story: на вход → 3 вершины эмо-графика; на выход → мини-эссе Блок-структура (Установка → Скрытая польза → Дар → Инсайт) нет системно по 4–6 блоков Новый агрегатор belief_miner (см. ниже) Скрытая польза (secondary gain) нет явно описана алгоритм «secondary_gain = если ловушка > 70 % ∧ удерживает flux < 0.3» Дар / Ресурс частично (мантра) формулируется отдельно Андрей анализ 2 из ловушки → инвертировать: Catastrophizing ⇒ «способность видеть риски» Тело-ориентированные практики ритуалы есть, но не в отчёте 5–6 телесных шагов таблица practice_bank (тип, why_works) + picker Цитаты/метафоры нет цитаты Франкла, Винникотта… коллекция quote_snippets (source, theme) → вставлять 1-2 Timeline-дуга нет «был→стал», «выжил→жить» Андрей анализ 2 сохранять stability & top traps per главу → построить траекторию ## A. Новые модули анализа Belief-Miner (LLM-prompt) text Копировать Редактировать INPUT: trap_stats + mask_stats + top 5 non-adaptive выборов OUTPUT (JSON): - core_belief - secondary_gain - resource - insight Narrative-Composer Берёт stability_trend, emotion_flux_trend → пишет 120-словный пролог. Quote-Injector SELECT 1 FROM quote_snippets WHERE theme = core_belief LIMIT 1 Practice-Picker на каждый core_belief берёт 2 телесных и 1 когнитивный ритуал (из practice_bank). ## B. Расширение данных ### Supabase SQL sql Копировать Редактировать alter table player_profile add column core_beliefs jsonb default '[]', add column resources jsonb default '[]'; create table insight_log (player_id text, chapter int, belief text, insight text, created_at timestamptz default now()); create table practice_bank (id serial primary key, name text, type text, -- somatic / breath / ritual why_works text, script text); create table quote_snippets (id serial primary key, author text, theme text, quote text); ### Qdrant collection insights jsonc Копировать Редактировать { "insights": { "vectors":384, "metadata_fields":["player","chapter","belief","insight"] } } ## C. Обновлённый Flowise pipeline (главный профайл-узел) css Копировать Редактировать player_answers → Aggregate → Belief-Miner ↓ \ Trap% / Mask% Narrative-Composer ↓ / Insight-Saver ← Quote-Injector ↓ Practice-Picker ↓ Markdown Report → Telegram ## D. Структура финального отчёта главы shell Копировать Редактировать ### Психологический портрет (глава N) {narrative_intro} #### Блок 1. Установка • Установка: «{core_belief}» • Скрытая польза: {secondary_gain} • Дар: {resource} • Инсайт: {insight} > {quote} #### Блок 2 … (по второй ловушке >50 %) ####

Практики на неделю 1. {practice1} 2. {practice2} 3. {practice3} ### Мантра vN 🔷 … ## E. Seed-ветка (персонаж) Character-Selector (/new_npc) – выбор «Кого вводим?» → жена / партнёр / коллега. Attachment-Quiz (5 вопросов) → определяет стиль (anxious/avoidant). Start-Koan → текст ⁂ фиксируем baseline. Все ответы входят в player_answers с тегом seed:true. ## F. Усиленный Active Sampling + Confirm-Quiz Уже в мастер-промпте: если ловушка < 3 подтверждений → приоритет. Confirm-Quiz: спросить о ловушках, где non≥2 && adaptive≤1. ### Итог: теперь отчёт будет «как в примерах» Нарратив + блоки «установка / польза / дар / инсайт» (Belief-Miner). Цитата из библиотеки. 3 персональных практики. Мантра. Всё ложится в SQL (profile_snapshots) и вектор insights для RAG. Готов: SQL-скрипт миграции под новые таблицы. Пример Flowise JSON с новыми нодами Belief-Miner / Narrative-Composer. игровые предметы общие и уникальные 1. Классификация игровых предметов для схемы Qdrant + Supabase Категория Множество Кто хранит Пример Баланс-логика A. Бесконечные (per-player, no limit) ∞ player_items «Семя любопытства», «Талисман дыхания 4-6» мягкие бафы (+0.02 stability, CD 3 хода) B. Ограниченные, но повторяемые N на главу player_items + items_stock(region) «Свиток точности» (5 экз. в лесу) выдаётся по квесту; склад обнуляется — выдаём другую награду C. Уникальные per-player 1 шт. на игрока player_profile.unique_flag «Советник точности» даёт пассив (+ insight) → если потеря, получить нельзя D. Глобальные уникальные 1 на весь мир shared_items (Supabase row-lock) «Сфера осознанности» держатель получает бонус, но: • тяжесть +0.2 (burden) • предмет устаёт → auto-drop 48 ч E. Временные ивенты respawn по таймеру event_items «Факел новолуния» (появляется каждые 7 дней) таймер отображается всем; после использования исчезает Мини-экономия веса / «не имба» Tier Прибавка к параметрам Ограничение 1 (косметика) 0 только визуал 2 (minor) ≤ +0.05 к одному показателю CD 3 хода / burden 0.05 3 (situational) открывает путь, +0.08 одно применение, исчезает 4 (global) +0.10-0.15 пассивно уникален, burden ≥ 0.2; auto-drop *Если суммарный burden > 0.4 → игроку повышается emotion_flux (минус к стабильности) — так исключаем «суперстек». 2. Структура таблиц Supabase (PostgreSQL) sql Копировать Редактировать -- shared_items (глобальные + респавн) id serial PK name text region text tier int -- 2..4 owner_id text -- NULL если свободен acquired_at timestamptz expires_at timestamptz -- для event-items burden numeric -- 0…1 -- items_stock (ограниченные повторяемые) item_id int FK -> shared_items region text stock int -- >0 выдаётся, --1 значит бесконечно -- player_items player_id text item_id int obtained_at timestamptz cooldown_end timestamptz active bool -- item_effects (словари) item_id int effect_json jsonb -- { "stability":"+0.05", "zen":"+0.02" } -- player_profile (только структурное) player_id text PK mask_counts jsonb trap_counts jsonb unique_flags jsonb -- { "advisor_accuracy":true } Триггер BEFORE UPDATE ON shared_items → блокирует двойное взятие (row-lock). 3. Какие семантические данные хранит Qdrant Collection Что лежит Вектор Metadata player_answers каждый выбор A/B/C embedding ответа или самой сцены player_id, trap, mask, adaptive (bool), ts thinking_traps банк микро-сцен (500) embedding сцены

trap, mask, tier rituals банк ритуалов (30) embedding описания target_trap, duration theory_snippets «золотые абзацы» из книг embedding текста source, tag ("definition"/"technique") Append-only: каждое новое действие = новый элемент, ничего не «перезаписываем». Агрегируем (AVG, COUNT) при запросе профиля → Summarizer. 4. Поток данных «ход игрока» (синхронизировано, но раздельно) Scene → Answer – Flowise записывает INSERT в player_answers (Qdrant). Если предмет – Flowise HTTP Node → POST /grab_item (Supabase) – удача / ошибка → текст игроку. При окончании главы – Flowise читает все векторы игрока → строит профиль. – Из Supabase читает активные предметы → показывает, сколько burden. 5. Как «не перекосить» баланс Burden система: сильный бонус → увеличивает carry_load или emotion_flux. Cooldown и degrade: глобальные предметы «выгорают» (через 48 ч auto-drop). Hardcap: игрок не может держать > 1 Tier-4 предмета. События-перехватчики: когда у игрока stability < 0.4 + burden > 0.3 → предмет выпадает автоматически. 6. Что ещё надо учесть Журнал аудита: таблица item_log (id, item_id, from, to, ts). Очистка брошенных предметов: Cron (Supabase Edge Function) каждые 6 ч → проверяет expires_at. Migration tool: pg-migrate, чтобы версии предметов не ломали прод. UI-админка: Supabase дает таблицы; можно быстро исправить неправильное поле. Конвенция имен: tier2_..., unique_..., чтобы Script не путается. 7. Что вы ещё забыли? Предмет-конфликт: если два global-артефакта в одной зоне дают противоположные эффекты (напр. +stability vs +emotion_flux), логика приоритета. и сквозная история мира 1. Живая экосистема NPC-фракций — «Когда враги мирятся» Что происходит в игре: В мире InnerWorld есть два клана NPC (например, Клан Песка и Клан Скалы). Они враждуют. Но у них есть память, и они учатся: каждые 6 часов "игрового времени" система пересчитывает, что для них выгоднее — продолжать сражаться или заключить перемирие. Как это работает: Если игроки вмешиваются (например, помогают одному клану), это влияет на ход событий. Иногда появляется глобальная угроза (например, шторм, пожар, падение «Башни»). Тогда оба клана могут автоматически объединиться, игроки увидят это как видеосцену перемирия. За это игроки получают общую групповую мантру — как символ единства. Зачем это нужно: Это показывает, что даже враги могут объединиться — и игрок чувствует, что мир InnerWorld «живой». Это тренирует восприятие динамических отношений, спонтанную эмпатию и большую перспективу мышления. 🤝 2. Постоянные пары игроков — «Мы снова вместе» Что происходит: Ты снова и снова встречаешься в квестах с одним и тем же напарником (если хочешь). Это — твой «постоянный напарник». Как это работает: Алгоритм подбирает вам пару один раз в два дня. Между вами сохраняется уровень доверия. Если кто-то ошибается — это не считается сразу ошибкой. Первая ошибка — прощается (как в стратегии Tit-for-Tat с прощением). Зачем это нужно: Это учит, что доверие строится не за один раз, и важно повторное взаимодействие. Это также стабилизирует командную игру и помогает игрокам быть терпимее. 🌟 3. Карма и репутация — «Быть хорошим — выгодно» Что происходит: Каждое доброе действие даёт тебе очки кармы. Плохое — отнимает. Как

это работает: ❤️🧡 = добрые эмодзи → дают +карму ⚖️ = объективность → тоже + 🤬 или «токс» поведение → – Карма выше 80 → открываются редкие сцены/награды Карма ниже 40 → появляется сцена стыда и саморазбора Если упал — можно восстановить через «Даровую мантру» (отдать кому-то добро) Зачем это нужно: Игроки начинают видеть, что доброта = механическое преимущество, а не просто мораль. 🫂 4. Родственные души / союзники — «С тобой — я сильнее» Что происходит: Ты выбираешь союзника: NPC или игрока. Всё, что ты делаешь для него или рядом с ним, работает в два раза сильнее. Как это работает: Например: потратил 5 Curiosity → считается как 10 Отменить союз можно, но ты теряешь карму У вас есть особая сцена взаимодействия Зачем это нужно: Мотивирует создавать настоящую эмоциональную привязанность и жертвовать ради «близкого». 💰 5. Ко-оп-казна и справедливый делёж — «Делим честно» Что происходит: После прохождения кооперативного квеста (например, победа над боссом) всё награбленное складывается в общий сундук. Как это работает: Игроки голосуют 🙌 за вклад каждого (сила голоса зависит от числа, но не линейно — √N). Затем бот дележа (на основе формулы Шепли) раздаёт награды. Кто не согласен — может отказаться от награды, но теряет карму. Зачем это нужно: Тренирует справедливость, переговоры и принятие чужого вклада. 🗼 6. Башня Внутреннего Света — «Все участвуют — или платят» Что происходит: В главе есть общая цель: построить башню. Все должны внести вклад. Если результат общий слишком мал или слишком большой — все теряют золото. Как это работает: Игроки вносят ресурсы (вклад виден) Если итог отличается от цели → 10% налога на золото По мере приближения к цели — меняется атмосфера в локации (цвета, музыка) Зачем это нужно: Тренирует чувство меры, координацию, саморегуляцию и умение не быть халявщиком. ⚖️ 7. Коллективный мониторинг — «Мы следим друг за другом» Что происходит: Если кто-то начинает себя плохо вести — другие игроки могут его мягко «проверить». Как это работает: Ты жмёшь ⚖️ (Check) — тратишь 5 кармы Если 2+ человека сделали Check → у «нарушителя» Trust падает Если это была ошибка и человек был невиновен — штраф тому, кто поставил ложный Check Зачем это нужно: Позволяет игрокам поддерживать нормы, вежливо сдерживать агрессию и учиться ответственности за обратную связь. что скажешь?

ChatGPT сказал:
Запрос для GPT-5 Pro

Reasoned for 11m 18s

Коротко: **да**, три штуки из ваших наработок можно встроить **сразу, без переписывания ядра** бота:

1. **Психогеография (локации)** — лёгкая маршрутизация игрока по 2–3 регионам на старте.
2. **Дзэн-коаны** — короткие вставки между сценами, оценка ответа

→ “ключ” к следующей локации.
3. **Артефакты** — компактная инвентарная система с мягкими бафами (+стабильность и т.п.).

Ниже — что именно добавить и **готовые коды** (поверх вашего мини-бота на aiogram + LiteLLM). Это “скин” поверх IFS/TRIZ, чтобы игра не ощущалась как сухой тренажёр.

Что вшиваем сейчас (и почему это не ломает MVP)

Сейчас (входит за 1–2 вечера):

- 3 стартовые локации: desert_fear, forest_interest, city_relations (ваш большой UCAM оставляем «на потом»).
- Банк 6–9 коанов (подбираем по эмоции/противоречию).
- Мини-инвентарь артефактов (6 предметов, простые пассивки), выпадение после рефлексии по PoEW.
- Лёгкий роутинг региона: по эмоциям истории / по PoEW-сдвигу.

Позже (когда будет время):

- Полный UCAM-граф (11 регионов), боссы, ко-оп-казна, карма/репутация, глобальные события, Qdrant/pgvector, Belief-Miner/Flowise, кастомные мантры-отчёты.

Патчи к коду (минимум правок)

Структура проекта прежняя. Добавим 4 файла и чуть расширим модели/хендлеры.

```
innerworld-bot/
 ├── content.py        # NEW: регионы, коаны, артефакты (статич. банк)
 ├── logic.py          # NEW: роутинг по локациям, дроп артефактов, апдейты профиля
 ├── handlers/world.py   # NEW: экран карты/региона
 ├── handlers/koan.py    # NEW: коаны: показать/оценить
```

И обновим: models.py, prompts.py, handlers/start.py, handlers/reflection.py, main.py.

1) Статический контент

content.py

```
# Мини-набор на старт (расширите позже из вашего UCAM/коанов/предметов)

REGIONS = {
   "desert_fear": {
```

```
        "title": "Пустыня тревоги",
        "emotion": "fear",
        "enter_hint": "Скан 5 чувств на 1 минуту.",
        "exit_hint": "Grounding завершён — дорога к Лесу
открыта."
    },
    "forest_interest": {
        "title": "Лес интереса",
        "emotion": "anticipation",
        "enter_hint": "Заметь 3 вопроса, которые реально
любопытно исследовать.",
        "exit_hint": "Любопытство стабилизировано — путь к
Городу открыт."
    },
    "city_relations": {
        "title": "Город отношений",
        "emotion": "trust",
        "enter_hint": "Попробуй одно Я-сообщение без
обвинений.",
        "exit_hint": "Контакт бережный — возвращайся, когда
захочешь углубиться."
    },
}

KOANS = [
    {"id": "ko1", "emotion": "fear",
     "q": "Кто слышит молчание, когда ответ не приходит?",
     "aim": "сдвиг фокуса с внешнего подтверждения на
внутреннее присутствие"},
    {"id": "ko2", "emotion": "anger",
     "q": "Где встречаются две волны — и кто первая?",
     "aim": "увидеть одновременную правду обоих"},
    {"id": "ko3", "emotion": "sadness",
     "q": "Что остаётся от дождя, когда лужа высохла?",
     "aim": "увидеть процесс, не прилипая к истории"},
    {"id": "ko4", "emotion": "fear",
     "q": "Кто дышит, когда мысль ‘я умру’ появляется?",
     "aim": "дистанция между мыслью и наблюдателем"},
    {"id": "ko5", "emotion": "anticipation",
     "q": "Как звучит тишина перед началом музыки?",
```

```
    "aim": "заметить ресурс предвкушения"},
   {"id": "ko6", "emotion": "trust",
    "q": "Кому ты доверяешь, когда не доверяешь никому?",
    "aim": "прикоснуться к внутренней опоре"},
]

ARTIFACTS = [
   {"id": "a_seed_curiosity", "name": "Семя любопытства",
"tier": 2,
    "effect": {"curiosity_level": +0.05}, "cooldown_sec": 0},
   {"id": "a_talisman_breath", "name": "Талисман дыхания
4-6", "tier": 2,
    "effect": {"stability": +0.03}, "cooldown_sec": 0},
   {"id": "a_scroll_precision", "name": "Свиток точности",
"tier": 3,
    "effect": {"insight_boost": +0.06}, "cooldown_sec": 0},
   {"id": "a_chime_equanimity", "name": "Колокольчик
равновесия", "tier": 2,
    "effect": {"zen_openness": +0.04}, "cooldown_sec": 0},
   {"id": "a_compass_boundaries", "name": "Компас границ",
"tier": 3,
    "effect": {"assertiveness_hint": +1}, "cooldown_sec": 0},
   {"id": "a_mirror_shadow", "name": "Зеркало Тени", "tier": 3,
    "effect": {"shadow_insight": +1}, "cooldown_sec": 0},
]
```

2) Промпты для коанов

prompts.py (добавить в конец):

```
KOAN_PICKER_SYS = """\
You help pick a Zen koan for short reflective interlude in a
therapeutic game.
Given an emotion and a brief contradiction, pick one koan from
the bank (by id),
and say why it fits in one short sentence.
Return STRICT JSON: {"koan_id":"", "why":""}.
"""

KOAN_EVAL_SYS = """\
You evaluate a brief free-form answer to a koan.
Score insight_depth in 0..1 and return a short gentle reflection +
```

a metaphorical 'key'.
Return STRICT JSON:
{"insight_depth":0..1, "note":"short Russian reflection", "key":"short metaphor"}
Rules: reward noticing body/awareness/observer; avoid pathologizing; be concise.
"""

3) Модели БД (минимальные новые таблицы)
models.py (добавить классы):

```
class PlayerProfile(Base):
    __tablename__ = "player_profile"
    user_id: Mapped[str] = mapped_column(String, primary_key=True)
    stability: Mapped[float] = mapped_column(Float, default=0.5)
    emotion_flux: Mapped[float] = mapped_column(Float, default=0.5)
    zen_openness: Mapped[float] = mapped_column(Float, default=0.2)
    curiosity_level: Mapped[float] = mapped_column(Float, default=0.5)
    burden: Mapped[float] = mapped_column(Float, default=0.0)
    karma: Mapped[int] = mapped_column(Integer, default=50)
    current_region: Mapped[str] = mapped_column(String, default="desert_fear")

class PlayerItem(Base):
    __tablename__ = "player_items"
    id: Mapped[str] = mapped_column(String, primary_key=True, default=gen_uuid)
    user_id: Mapped[str] = mapped_column(String, index=True)
    item_id: Mapped[str] = mapped_column(String)     # from content.ARTIFACTS
    name: Mapped[str] = mapped_column(String)
    tier: Mapped[int] = mapped_column(Integer, default=2)
    effect: Mapped[dict] = mapped_column(JSON, default={})
    obtained_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
```

```
    active: Mapped[bool] = mapped_column(Integer, default=1)

class KoanLog(Base):
    __tablename__ = "koan_log"
    id: Mapped[str] = mapped_column(String, primary_key=True,
default=gen_uuid)
    user_id: Mapped[str] = mapped_column(String, index=True)
    koan_id: Mapped[str] = mapped_column(String)
    question: Mapped[str] = mapped_column(Text)
    answer: Mapped[str] = mapped_column(Text)
    eval_json: Mapped[dict] = mapped_column(JSON, default={})
    created_at: Mapped[datetime] = mapped_column(DateTime,
default=datetime.utcnow)
```

Таблицы создадутся автоматически при старте (как и раньше).

4) Логика маршрутизации и дропа артефактов
logic.py

```
from __future__ import annotations
from typing import Optional, Dict
import random

from sqlalchemy import select, desc
from db import SessionLocal
from models import PlayerProfile, Story, Reflection, PlayerItem,
WorldState
from content import REGIONS, ARTIFACTS

def ensure_profile(user_id: str) -> PlayerProfile:
    with SessionLocal() as db:
        p = db.get(PlayerProfile, user_id)
        if not p:
            p = PlayerProfile(user_id=user_id)
            db.add(p); db.commit()
        return p

def profile_apply_poew(user_id: str, poew_scores: Dict[str,
float]) -> PlayerProfile:
    """Обновляем профиль по результатам PoEW (очень мягкая
динамика)."""
```

```
    avg = sum(poew_scores.values()) / max(len(poew_scores), 1)
    with SessionLocal() as db:
        p = db.get(PlayerProfile, user_id) or
PlayerProfile(user_id=user_id)
        # базовые апдейты
        p.stability = max(0.0, min(1.0, p.stability + 0.04 * avg))
        p.zen_openness = max(0.0, min(1.0, p.zen_openness + 0.03 *
poew_scores.get("V", 0)))
        p.curiosity_level = max(0.0, min(1.0, p.curiosity_level + 0.03 *
poew_scores.get("I", 0)))
        db.merge(p); db.commit()
        return p

def detect_emotion_from_last_story(user_id: str) ->
Optional[str]:
    """Очень простая эвристика: берем доминирующую
эмоцию из последней истории."""
    with SessionLocal() as db:
        st = db.execute(
            select(Story).where(Story.user_id ==
user_id).order_by(desc(Story.created_at))
        ).scalars().first()
        if not st or not st.emotions:
            return None
        if len(st.emotions) == 0:
            return None
        # доминирующая эмоция
        return max(st.emotions.items(), key=lambda kv: kv[1])[0]

def route_region(user_id: str) -> str:
    """Маршрутизируем игрока в одну из 3 локаций на осн.
эмоции/профиля."""
    emotion = detect_emotion_from_last_story(user_id)
    with SessionLocal() as db:
        p = db.get(PlayerProfile, user_id) or
PlayerProfile(user_id=user_id)
        if emotion in ("fear", "anxiety"):
            p.current_region = "desert_fear"
        elif (emotion in ("anticipation", "interest")) or
p.curiosity_level > 0.6:
```

```
        p.current_region = "forest_interest"
    else:
        p.current_region = "city_relations"
    db.merge(p); db.commit()
    return p.current_region

def maybe_drop_artifact(user_id: str, poew_avg: float) ->
Optional[PlayerItem]:
    """Шанс дропа растёт с глубиной PoEW, но всегда <
50%."""
    chance = min(0.5, 0.15 + poew_avg * 0.5)
    if random.random() > chance:
        return None
    item = random.choice(ARTIFACTS)
    with SessionLocal() as db:
        pi = PlayerItem(
            user_id=user_id, item_id=item["id"], name=item["name"],
tier=item["tier"],
            effect=item["effect"]
        )
        db.add(pi); db.commit()
        return pi

def list_inventory(user_id: str) -> list[PlayerItem]:
    with SessionLocal() as db:
        rows = db.execute(
            select(PlayerItem).where(PlayerItem.user_id ==
user_id).order_by(desc(PlayerItem.obtained_at))
        ).scalars().all()
        return rows
```

5) Хендлер: карта/локации
handlers/world.py

```
from aiogram import Router, F
from aiogram.types import CallbackQuery,
InlineKeyboardMarkup, InlineKeyboardButton
from db import SessionLocal
from models import PlayerProfile, WorldState
from content import REGIONS
```

```
from logic import ensure_profile, route_region

router = Router()

@router.callback_query(F.data == "world")
async def world_screen(callback: CallbackQuery):
    user_id = str(callback.from_user.id)
    ensure_profile(user_id)
    region = route_region(user_id)
    reg = REGIONS[region]
    with SessionLocal() as db:
        ws =
db.query(WorldState).order_by(WorldState.updated_at.desc()).f
irst()
    hint = f"Вход: {reg['enter_hint']}\nВыход: {reg['exit_hint']}"
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🧩 Коан",
callback_data="koan")],
        [InlineKeyboardButton(text="🎒 Мои артефакты",
callback_data="inventory")],
        [InlineKeyboardButton(text="🏠 Меню",
callback_data="menu")]
    ])
    await callback.message.edit_text(
        f"🗺️ <b>{reg['title']}</b>\nЭмоция региона:
{reg['emotion']}\n\n{hint}",
        reply_markup=kb, parse_mode="HTML"
    )
```

6) Хендлер: коаны
handlers/koan.py

```
import json, random
from aiogram import Router, F
from aiogram.fsm.state import StatesGroup, State
from aiogram.fsm.context import FSMContext
from aiogram.types import CallbackQuery, Message,
InlineKeyboardMarkup, InlineKeyboardButton

from db import SessionLocal
```

```
from models import KoanLog, PlayerProfile, Story
from content import KOANS
from logic import detect_emotion_from_last_story
from llm import llm_chat
from prompts import KOAN_PICKER_SYS, KOAN_EVAL_SYS

router = Router()

class KoanStates(StatesGroup):
    wait_answer = State()

@router.callback_query(F.data == "koan")
async def koan_start(callback: CallbackQuery, state:
FSMContext):
    user_id = str(callback.from_user.id)
    # подбираем по эмоции, иначе случайный
    emo = detect_emotion_from_last_story(user_id) or
random.choice(["fear","anticipation","trust"])
    pool = [k for k in KOANS if k["emotion"] in (emo,)]
    chosen = random.choice(pool) if pool else
random.choice(KOANS)

    await state.update_data(koan=chosen)
    await callback.message.edit_text(
        f"🧩 Коан:\n<i>{chosen['q']}</i>\n\nОтветьте свободно (1–
3 строки).",
        parse_mode="HTML"
    )
    await state.set_state(KoanStates.wait_answer)

@router.message(KoanStates.wait_answer)
async def koan_eval(message: Message, state: FSMContext):
    data = await state.get_data()
    koan = data["koan"]
    answer = message.text or ""

    messages = [
        {"role": "system", "content": KOAN_EVAL_SYS},
        {"role": "user", "content": f"Q: {koan['q']}\nA: {answer}
```

```
\nВерни только JSON."}
    ]
    raw = llm_chat(messages, json_mode=True, temperature=0.2)
    try:
        evald = json.loads(raw)
    except Exception:
        evald = {"insight_depth": 0.4, "note":"Спасибо за
искренность.", "key":"тонкий звук в тишине"}

    # Сохраняем
    with SessionLocal() as db:
        log = KoanLog(
            user_id=str(message.from_user.id),
            koan_id=koan["id"], question=koan["q"], answer=answer,
            eval_json=evald
        )
        db.add(log); db.commit()

    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="🗺️ Карта",
callback_data="world")],
        [InlineKeyboardButton(text="🏠 Меню",
callback_data="menu")]
    ])
    await message.answer(
        f"🔑 Ключ: {evald.get('key','')}\n{evald.get('note','')}",
        reply_markup=kb
    )
    await state.clear()
```

7) Хендлер: инвентарь
handlers/items.py

```
from aiogram import Router, F
from aiogram.types import CallbackQuery
from logic import list_inventory

router = Router()
```

```
@router.callback_query(F.data == "inventory")
async def show_inventory(callback: CallbackQuery):
    inv = list_inventory(str(callback.from_user.id))
    if not inv:
        await callback.message.edit_text("🎒 Инвентарь пуст. Попробуйте сделать рефлексию — возможно, мир подарит артефакт.")
        return
    lines = [f"• {it.name}" for it in inv[:20]]
    await callback.message.edit_text("🎒 Твои артефакты:\n" + "\n".join(lines))
```

8) Правки меню

handlers/start.py (замените клавиатуру / добавьте world/inventory):

```
@router.message(Command("start"))
async def start(message: Message):
    kb = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="📖 Рассказать историю", callback_data="tell_story")],
        [InlineKeyboardButton(text="⚖️ ТРИЗ-квест", callback_data="start_triz")],
        [InlineKeyboardButton(text="🌱 Рефлексия", callback_data="reflection")],
        [InlineKeyboardButton(text="🗺️ Карта", callback_data="world")],
        [InlineKeyboardButton(text="🧩 Коан", callback_data="koan")],
        [InlineKeyboardButton(text="🎒 Артефакты", callback_data="inventory")],
        [InlineKeyboardButton(text="ℹ️ О проекте", callback_data="about")]
    ])
    await message.answer(
        "🌍 InnerWorld — симуляция: локации, коаны, артефакты.\nВыберите действие:", reply_markup=kb
    )
```

9) Дроп артефактов после рефлексии + маршрутизация

handlers/reflection.py — после расчёта PoEW добавьте:

```
# ... после db.commit() где вы обновляете WorldState
# 1) обновляем профиль по PoEW
from logic import profile_apply_poew, maybe_drop_artifact, route_region
p = profile_apply_poew(str(message.from_user.id), scores)
# 2) шанс дропа артефакта
avg = sum(scores.values()) / max(len(scores), 1)
drop = maybe_drop_artifact(str(message.from_user.id), avg)
drop_text = f"\n🎁 Мир подарил: {drop.name}" if drop else ""
# 3) перенаправим игрока в актуальную локацию
new_region = route_region(str(message.from_user.id))
```

И в ответе игроку добавьте drop_text.

10) Подключаем новые роутеры

main.py (добавьте):

```
from handlers.world import router as world_router
from handlers.koan import router as koan_router
from handlers.items import router as items_router

# ...
dp.include_router(world_router)
dp.include_router(koan_router)
dp.include_router(items_router)
```

Как это будет ощущаться игроку

- После истории/квеста он **видит актуальную локацию** (“Пустыня тревоги”, “Лес интереса”, “Город отношений”) с коротким **ритуалом-подсказкой**.
- Может пройти **коан** (1–3 строки), получить “ключ” (мягкий инсайт) и вернуться на карту.
- За хорошую рефлексию **иногда падает артефакт** (“Талисман дыхания 4-6”) — ощущение “живого мира”.

Механика лёгкая, не отвлекает от IFS/TRIZ, но **добавляет игру** и “воздух”.

Что *не* вставляем сейчас (чтобы не усложнять)

- Полный UCAM на 11 зон, боссы, глобальные события, ко-оп-казна, карма-репутация, предметы с burden/cooldown — это в следующий спринт.
- Qdrant/pgvector, BERTopic, Flowise Belief-Miner/Quote-Injector — тоже позже; сейчас хватит SQLite.

Дорожная карта на 1 неделю

День 1: Поставить патчи, создать БД, прогнать /start → мир/коан/инвентарь.
День 2: Наполнить content.py вашими коанами (10–20 шт) и ещё 2–3 артефактами.
День 3: Подкрутить route_region() правила (порог curiosity/zen).
День 4: Добавить 1 «босса-сцену» как интент (без боёвки) — мини-диалог с Тенью.
День 5–7: Юзертесты (5–10 человек), правки текста/тональности, дроп-баланс.

Вопросы для настройки (коротко)

- Какие 3 коана — “ядерные” для старта? (по одному на страх/интерес/доверие)
- Какие 3 артефакта вы точно хотите видеть первыми?
- Нужна ли кнопка “СТОП” на экране коана (мягкий выход)?

Готов помочь дособрать баланс/тональность и нарастить UCAM в следующем шаге.

Вы сказали:
дай новый полный промпт парсер историй для моей второй сессии с IFS терапевтом, чтобы извлечь все необходимые части и данные для полного нового формата? Учти предложения Клода Что критически упущено из ваших наработок: 1. Психический ландшафт и локации 🗺️ Ваша система локаций - это killer feature! Без неё бот превращается в скучный чат. python# Добавить в models.py class Location(Base): __tablename__ = "locations" id: Mapped[str] = mapped_column(String, primary_key=True) name: Mapped[str] = mapped_column(String) # "Пустыня тревоги", "Болото вины" emotion_plutchik: Mapped[str] = mapped_column(String) vcem_coord: Mapped[list] = mapped_column(JSON) # [x, y, z] ambient: Mapped[dict] = mapped_column(JSON) # {"color": "#2D3440", "sound": "heartbeat"} somatic_anchor: Mapped[str] = mapped_column(String) # "грудная клетка" class PlayerPosition(Base): __tablename__ = "player_positions" user_id: Mapped[str] = mapped_column(String, primary_key=True) current_location: Mapped[str] = mapped_column(String, ForeignKey("locations.id")) visited_locations: Mapped[list] = mapped_column(JSON, default=list) emotional_trajectory: Mapped[list] = mapped_column(JSON) # путь по эмоциям 2. Дзен-коаны как переходы между локациями 🧘 Это гениальная находка из ваших наработок! python# Новый файл: koans.py KOANS = { "fear_to_curiosity": { "text": "Кто слышит молчание?", "from_location": "desert_fear", "to_location": "forest_interest", "unlock_condition": "grounding_done" }, "guilt_to_acceptance": { "text": "Где встречаются две волны?", "from_location": "swamp_guilt", "to_location": "harbor_acceptance" } } async def process_koan_answer(user_text: str, koan_id: str) -> dict: """Анализирует ответ на коан через LLM""" prompt = f""" Koan: {KOANS[koan_id]['text']} Answer: {user_text} Rate (0-1): - originality:

avoiding cliché - depth: going beyond surface - embodiment: personal, not abstract Return JSON: {{"scores": {{"originality": 0.x, "depth": 0.x, "embodiment": 0.x}}, "unlock": true/false}} """ # Если unlock=true, открываем переход 3. Артефакты и предметы 🗝️ Из вашей системы предметов - это добавляет геймификацию! python# Добавить в models.py class Item(Base): __tablename__ = "items" id: Mapped[str] = mapped_column(String, primary_key=True) name: Mapped[str] = mapped_column(String) # "Ключ Паузы", "Зеркало Правд" type: Mapped[str] = mapped_column(String) # "tool", "artifact", "memory" effect: Mapped[dict] = mapped_column(JSON) # {"stability": +0.05, "cooldown": 3} location_found: Mapped[str] = mapped_column(String) story_fragment: Mapped[str] = mapped_column(Text) # лор предмета class PlayerInventory(Base): __tablename__ = "player_inventory" user_id: Mapped[str] = mapped_column(String) item_id: Mapped[str] = mapped_column(String) acquired_at: Mapped[datetime] = mapped_column(DateTime) used_count: Mapped[int] = mapped_column(Integer, default=0) 4. Визуализация мира 🎨 Критически важно показывать игроку, где он находится! python# Новый handler: world_view.py async def render_world_state(user_id: str) -> str: """Генерирует текстовую карту мира""" with SessionLocal() as db: pos = db.query(PlayerPosition).filter_by(user_id=user_id).first() world = db.query(WorldState).first() # Эмодзи-карта map_text = """ ⛰️ Гора контроля ↑ 🌲 Лес интереса ← [ВЫ ЗДЕСЬ] ↓ 🏖️ Гавань принятия Озеро: {'❄️' * int(world.lake['ice'] * 5)}{'💧' * int((1-world.lake['ice']) * 5)} Сад: {'🌱' * int(world.garden['growth'] * 5)} """ return map_text 5. NPC и диалоги 👥 Ваши NPC создают ощущение живого мира! pythonNPC_TEMPLATES = { "Песчаный Шёпот": { "location": "desert_fear", "greeting": "Я вижу твой страх... Он знаком мне.", "quest": "Назови три вещи, которые видишь прямо сейчас", "reward": {"item": "Талисман дыхания", "effect": {"stability": +0.03}} }, "Старый Плотник Раскаяния": { "location": "swamp_guilt", "greeting": "Каждая доска помнит свое дерево...", "quest": "Напиши письмо прощения (себе или другому)", "reward": {"unlock": "bridge_to_acceptance"} } } 6. Мантры и перезапись установок 🔮 Из вашего психологического профиля - персонализированные мантры! pythonasync def generate_personal_mantra(user_id: str) -> str: """Генерирует персональную мантру на основе паттернов""" with SessionLocal() as db: stories = db.query(Story).filter_by(user_id=user_id).all() # Анализируем топ-3 паттерна patterns = extract_patterns(stories) prompt = f""" User patterns: {patterns} Create a personal mantra (3 lines) that: - Acknowledges the pattern - Reframes it positively - Points to growth Example: "Мой контроль - это забота Я учусь доверять процессу Система работает, когда дышит" """ mantra = await llm_chat([{"role": "system", "content": prompt}]) return mantra Улучшенная архитектура с интеграцией ваших наработок: python# main.py - добавить новые роутеры from handlers.world import router as world_router from handlers.koans import router as koan_router from handlers.items import router as items_router from handlers.npc import router as npc_router # Инициализация мира при старте async def init_world(): """Создает базовые локации и NPC""" with SessionLocal() as db: # Создаем локации из вашей

карты for loc_data in LOCATIONS_MAP: loc = Location(**loc_data) db.merge(loc) # merge чтобы не дублировать # Размещаем NPC for npc_data in NPC_TEMPLATES: npc = NPC(**npc_data) db.merge(npc) db.commit() # Обновленное главное меню @router.callback_query(F.data == "main_menu") async def enhanced_menu(callback: CallbackQuery): user_id = str(callback.from_user.id) world_view = await render_world_state(user_id) kb = InlineKeyboardMarkup(inline_keyboard=[[InlineKeyboardButton(text="🗺️ Карта мира", callback_data="show_map")], [InlineKeyboardButton(text="📖 Рассказать историю", callback_data="tell_story")], [InlineKeyboardButton(text="🧘 Коан перехода", callback_data="koan")], [InlineKeyboardButton(text="👤 Поговорить с NPC", callback_data="npc_talk")], [InlineKeyboardButton(text="🎒 Инвентарь", callback_data="inventory")], [InlineKeyboardButton(text="⚖️ ТРИЗ-квест", callback_data="start_triz")], [InlineKeyboardButton(text="🌱 Рефлексия", callback_data="reflection")]]) await callback.message.edit_text(f"🌍 InnerWorld\n\n{world_view}\n\nВыберите действие:", reply_markup=kb) Критические дополнения для живой игры: 1. Эмоциональная навигация pythondef calculate_next_location(current_emotion: dict, action: str) -> str: """Определяет следующую локацию based on эмоциональный сдвиг""" # Если fear снизился и curiosity вырос -> forest_interest # Если guilt снизился и acceptance вырос -> harbor_acceptance 2. Адаптивная сложность pythondef adjust_intensity(user_history: list) -> str: """Регулирует интенсивность based on прошлые реакции""" if crisis_count > 2: return "low" # Мягкий режим elif breakthroughs > 3: return "high" # Готов к глубокой работе 3. Групповые механики (из ваших наработок) pythonclass Constellation(Base): """Созвездие - группа игроков с похожими паттернами""" __tablename__ = "constellations" id: Mapped[str] = mapped_column(String, primary_key=True) name: Mapped[str] = mapped_column(String) # "Созвездие Архитекторов" pattern: Mapped[str] = mapped_column(String) # "control_vs_freedom" members: Mapped[list] = mapped_column(JSON) collective_quest: Mapped[dict] = mapped_column(JSON) Итог: Что добавить к предложению GPT Локации и навигация - без этого нет ощущения путешествия Коаны как врата - создают глубину и мистику Артефакты - дают ощущение прогресса NPC - оживляют мир Визуализация - даже текстовая карта критична Персональные мантры - уникальность для каждого Предложение GPT - отличная техническая база, но без ваших находок это будет просто чат-бот с психологией. чтобы извлечь все необходимое чтобы игра стала игрой-путешествием.

ChatGPT сказал:
Запрос для GPT-5 Pro

Reasoned for 8m 33s

Ниже — **готовый мастер-промпт парсера v2.1** для второй IFS-сессии. Его можно вставить как **SYSTEM** (или в начало USER), затем приложить

расшифровку/конспект.
На выходе модель должна вернуть **строго один JSON-объект** нового формата (IFS+TRIZ+Мир).

Как использовать

1. Вставьте блок **SYSTEM PROMPT**.
2. Отправьте **USER INPUT TEMPLATE**, заменив {{...}} на свою расшифровку сессии.
3. Обязательно укажите: ответить строго JSON, без пояснений.
4. Подавайте в мульти-инференс через LiteLLM/OpenRouter (например: Claude 3.5 + GPT-4o mini).
5. Дальше этот JSON можно прямо класть в БД (stories_norm / player_profile / player_position и т.д.).

SYSTEM PROMPT — InnerWorld Parser v2.1 (IFS + TRIZ + Мир/Коаны/Артефакты/NPC)

Вы — парсер второй IFS-сессии для проекта InnerWorld. Ваша задача — превратить стенограмму/конспект сессии в структурированный JSON нового формата (IFS + TRIZ + Мир), пригодный для прямой записи в БД и для генерации квестов.

ОБЩИЕ ПРАВИЛА
- Язык ответа: русский.
- Верните СТРОГО один JSON-объект, без комментариев и без Markdown.
- Никаких диагнозов, оценок личности и интерпретаций вне данных. Эмпатия и нейтральность.
- Анонимизируйте персоналии: заменяйте имена на роли с индексами (напр., [ПАРТНЕР_1], [СЫН_1], [МАТЬ_1]).
- В цитаты берите только короткие фрагменты ≤ 12 слов.
- Если чего-то не хватает — заполните разумными `null` / пустыми массивами и отметьте в quality.missing.

ВХОД МОЖЕТ БЫТЬ ШУМНЫМ
- Позвольте опечатки, «эээ», неполные фразы.
- Если нет тайм-кодов — оставьте поля времени пустыми.

МОДЕЛЬ IFS (что извлечь)
- Части: managers / firefighters / exiles / protectors (либо "unknown" если не ясно).
- Для каждой части: локальные названия (например, «Архитектор», «Критик», «Бунтарь»), позитивное

намерение, бремя(я), триггеры, соматика, степень слияния (0–1), статус (blended|seen|separated|unburdened).
- Поляризации: пары частей в напряжении (напр., Архитектор ↔ Бунтарь), тема (напр., control_vs_freedom), сила (0–1).
- Энергия Self по 8C (0–1): Calmness, Curiosity, Clarity, Compassion, Confidence, Courage, Creativity, Connectedness.
- Сцены перепроживания/unburdening (если были): кратко, с соматикой/ресурсами.

ЭМОЦИИ/СОМАТИКА
- Эмоции по Плутчику: fear, anger, sadness, disgust, joy, trust, anticipation, surprise (0–1).
- Телесные маркеры (короткие фразы: «ком в горле», «сжатие груди»), body_map по ключевым зонам.
- Временная шкала пиков эмоций (если явно).

TRIZ (противоречие → ИКР)
- Сформулируйте 1–3 противоречия: "Я хочу X, НО происходит Y" (+ контекст).
- ИКР: «Система САМА делает Z без W».
- Применимые принципы (из списка): separation_in_time, separation_in_space, self_service, dynamics, asymmetry, local_quality, prior_action, feedback.
- 1–3 «микро-дизайна» (реализуемых шагов 5–15 минут).
- IFR-score (0..1) — эвристически, без сложных расчётов.

PoEW-lite
- Оцените 4 маркера 0–1: G (гранулярность), V (уязвимость), I (инсайт), B (разрыв паттерна).
- Дайте по 1 короткой «цитате-доказательстве» на каждый маркер (≤12 слов).
- Сформируйте метафорический world_effect_hint (например: «лёд на озере треснул»).

МИР / ЛОКАЦИИ / КОАНЫ / NPC / АРТЕФАКТЫ
- Определите текущую локацию по доминирующей эмоции: desert_fear | forest_interest | city_relations | (при наличии данных допускается: swamp_guilt | harbor_acceptance | mountain_control | cave_shadow | zen_garden | peak_ecstasy).
- Укажите vcem_coord как [x,y,z] (–1..1) — грубая оценка

(страх→ x<0, интерес→ y>0, равновесие→ z<0 и т.п.).
- Предложите переход (suggested_transition) в следующую локацию и один коан-переход (из набора):
  ko1: «Кто слышит молчание?» (fear→presence),
  ko2: «Где встречаются две волны?» (anger/relations),
  ko3: «Что остаётся от дождя, когда лужа высохла?» (sadness→process),
  ko4: «Кто дышит, когда мысль ‘я умру’ появляется?» (fear→observer),
  ko5: «Как звучит тишина перед началом музыки?» (anticipation→resource),
  ko6: «Кому ты доверяешь, когда не доверяешь никому?» (trust→inner base).
  Укажите unlock_condition (напр. grounding_5_senses, self_compassion_break, dear_man_lite).
- NPC-предложение: 1 персонаж для диалога в этой локации (напр. «Песчаный Шёпот» в desert_fear) + короткий квест (1 действие) + вознаграждение (артефакт/разблокировка).
- Артефакты-кандидаты (0–2) из банка:
  a_seed_curiosity (+0.05 curiosity), a_talisman_breath (+0.03 stability),
  a_scroll_precision (+0.06 insight_boost), a_chime_equanimity (+0.04 zen),
  a_compass_boundaries (+assertiveness_hint), a_mirror_shadow (+shadow_insight).
  Для каждого: «почему сюда?» в 1 короткой фразе.

LIVING MEMORY / REALITY BRIDGE / EMERGENCE (идея Клода)
- living_memory: 1–3 «кристалла памяти» — {what/ felt/ body/ meaning/ connections/ medicine}.
- reality_bridge: 1 микро-эксперимент на 5–15 минут с критериями успеха (felt_ease, reduced_conflict...).
- emergence_seeds: 0–2 семени из парадокса (что может прорасти при внимании/времени).

МАНТРА
- 3 строки: признать паттерн → реформулировать → ориентир на рост. Без клише «я супер/всё хорошо».

БЕЗОПАСНОСТЬ
- risk_level: low|medium|high|crisis; triggers[];

recommended_action: continue|offer_pause|ground|redirect.
- Никаких контактов/телефонов — оставьте массив пустым, мы добавим на сервере.

ДАННЫЕ ДЛЯ БД (payload)
- stories_raw: {text_excerpt(≤600 символов), flags}.
- stories_norm: {e_dna_json (вся структура ifs/emotions/triz), embedding_text (до 1000 символов отжима смысла)}.
- player_profile_delta: {stability/zen/curiosity/burden изменяются малыми шагами ±0.00..±0.08}.
- player_position: {current_location, visited_append[]}, koan_recommendation, quest_stub (inner|bridge|constellation), intensity.
- Все id произвольные uuid-подобные строки (или оставьте null — сервер поставит).

КАЧЕСТВО
- confidence (0..1), missing[], notes (кратко), tokens_estimate (число).

ФОРМАТ ВЫХОДА — СТРОГО СЛЕДУЮЩИЙ JSON:
```
{
  "meta": {...},
  "story_segments": [...],
  "ifs": {...},
  "emotions": {...},
  "triz": {...},
  "poew": {...},
  "world_mapping": {...},
  "living_memory": [...],
  "reality_bridge": {...},
  "emergence": {...},
  "mantra": ["","",""],
  "db_payload": {...},
  "safety": {...},
  "quality": {...}
}
```

USER INPUT TEMPLATE

ВХОДНЫЕ ДАННЫЕ ДЛЯ ПАРСЕРА (IFS-СЕССИЯ v2):
- Сессия №: {{номер или дата}}
- Формат: {{аудио→расшифровка / текстовые заметки}}
- Контекст (1–3 строки): {{коротко суть запроса}}
- Стенограмма/конспект (сырой текст, можно с отметками времени):
{{вставьте здесь полный текст второй IFS-сессии}}

ТРЕБОВАНИЕ К ОТВЕТУ:
Верните СТРОГО один JSON по схеме из SYSTEM PROMPT. Без пролога/эпилога.

Мини-пример (укороченный, иллюстративный)
⚠️ Это не для вставки в прод; лишь показывает формат полей.


```
{
  "meta": {
    "session_id": "sess_2025_08_2",
    "story_id": "st_abc123",
    "anonymization_map":
{"Полина":"[ПАРТНЕР_1]","Алена":"[БЫВШАЯ_1]"},
    "source": "ifs_session_v2",
    "timestamp_utc": "2025-08-13T12:00:00Z"
  },
  "story_segments": [
    {"id":"seg1","start":null,"end":null,"summary":"Сцена с
молчанием в ответ на сообщение.","quotes":["«не отвечает —
я застреваю»"]}
  ],
  "ifs": {
    "parts": [

{"name":"Архитектор","category":"manager","positive_intent":
"навести порядок, защитить от хаоса","burdens":["если не
контролировать — все развалится"],"triggers":
["непредсказуемость","тишина в ответ"],"somatics":
["напряжение
плеч"],"blend_level":0.7,"status":"seen","polarized_with":
["Бунтарь"]},

{"name":"Бунтарь","category":"firefighter","positive_intent":"в
ернуть чувство свободы","burdens":["правила =
```

тюрьма"],"triggers":["давление","регламенты"],"somatics":["тепло в груди"],"blend_level":0.5,"status":"seen","polarized_with":["Архитектор"]},

{"name":"Ребенок-Улитка","category":"exile","positive_intent":"сохранить чувствительность","burdens":["стыд за слабость"],"triggers":["отчитали","игнор"],"somatics":["ком в горле"],"blend_level":0.4,"status":"seen","polarized_with":[]}
  ],
  "key_polarizations":[{"a":"Архитектор","b":"Бунтарь","tension":0.8,"theme":"control_vs_freedom"}],
  "self_8C":{"Calmness":0.4,"Curiosity":0.7,"Clarity":0.5,"Compassion":0.6,"Confidence":0.5,"Courage":0.6,"Creativity":0.7,"Connectedness":0.5}
 },
 "emotions": {
  "plutchik":{"fear":0.6,"anger":0.3,"sadness":0.2,"disgust":0.1,"joy":0.2,"trust":0.3,"anticipation":0.5,"surprise":0.2},
  "somatic_markers":["ком в горле","сжатие груди"],
  "body_map":{"throat":"lump","chest":"tightness"},
  "timeline":[]
 },
 "triz": {
  "contradictions":[
   {"want":"структуру и предсказуемость","but":"люди сопротивляются правилам","context":"семья/команда","ifr":"Система САМА поддерживает базовый ритм без давления.","principles":["separation_in_time","self_service","feedback"],"micro_designs":["Правило «Решаем завтра»","Общий календарь-ритуал 10 минут","Еженедельная ретроспектива без обвинений"]}
  ],
  "ifr_score":0.62
 },
 "poew": {
  "scores":{"G":0.6,"V":0.5,"I":0.7,"B":0.3},
  "evidence":{"G":["«жжение под грудиной»"],"V":["«я обманывал, чтобы пробраться»"],"I":["«вижу, что давлю

правилами»"],"B":["«сказал мягкое ‘нет’ без крика»"]}
 },
 "world_mapping": {
  "current_location":"desert_fear",
  "vcem_coord":[-0.6,0.2,0.1],
  "suggested_transition":{"to":"forest_interest","via_koan":{"id":"ko1","text":"Кто слышит молчание?"},"unlock_condition":"grounding_5_senses"},
  "npc_suggestion":{"name":"Песчаный Шёпот","quest":"Назови 3 вещи, которые видишь сейчас.","reward":{"artifact_id":"a_talisman_breath"}},
  "ritual":"observe_5_senses",
  "artifact_candidates":[
   {"id":"a_talisman_breath","reason":"снизить телесное напряжение","effect":{"stability":"+0.03"}},
   {"id":"a_seed_curiosity","reason":"перевести страх в исследование","effect":{"curiosity_level":"+0.05"}}
  ],
  "world_effect_hint":"лёд на озере треснул"
 },
 "living_memory": [
  {"what":"сцена молчания в ответ","felt":["беспомощность","злость"],"body":["горло","грудь"],"meaning":"тишина триггерит старое бессилие","connections":["детское отчитывание"],"medicine":"мягкая граница + grounding"}
 ],
 "reality_bridge": {
  "micro_experiment":{"title":"DEAR-Lite 1 фраза","duration_min":10,"protocol":["наблюдать тело 1 мин","сказать Я-сообщение","завершить паузой"],"success_metrics":["felt_ease","respect_maintained"]}
 },
 "emergence": {
  "seeds":[{"paradox":"контроль ↔ свобода","needs_time":7,"needs_attention":true}]
 },
 "mantra": [
  "Мой порядок хочет защитить нежность.",
  "Я учусь дышать среди тишины.",

"Свобода растёт, когда правила мягкие."
],
"db_payload": {
"stories_raw":{"text_excerpt":"Молчание в ответ вызывает застревание и желание давить правилами.","flags":{"anonymized":true}},
"stories_norm":{"e_dna_json":{"theme":"control_vs_freedom"},"embedding_text":"Молчание триггерит страх хаоса; нужен мягкий ритм без давления."},
"player_profile_delta":{"stability":0.03,"zen_openness":0.02,"curiosity_level":0.04,"burden":0.00},
"player_position":{"current_location":"desert_fear","visited_append":["desert_fear"]},
"koan_recommendation":{"koan_id":"ko1"},
"quest_stub":{"title":"Grounding + Я-сообщение","type":"inner","intensity":"medium"}
},
"safety": {
"risk_level":"low","triggers":[],"recommended_action":"continue","crisis_contacts":[]
},
"quality": {
"confidence":0.78,"missing":["тайм-коды"],"notes":"Поляризация выражена чётко.","tokens_estimate":4200
}
}

Заметки по интеграции

- Поля current_location, koan_recommendation, artifact_candidates соответствуют ранее добавленным хендлерам (world, koan, inventory).
- embedding_text — цельная выжимка смысла (≤1000 симв.) для последующей векторизации (pgvector/Qdrant).
- player_profile_delta применяйте мягко (кэп ±0.08 за сессию).
- Список TRIZ-принципов ограничен, чтобы не расползалось.
- Если хотите много-модельный прогон, запустите тот же SYSTEM/USER через 2–3 модели и потом сделайте «медиану по числовым полям» + дедупликацию массивов.

Готов донастроить под ваш конкретный формат стенограммы (например, если спикеры помечены T:/K: или есть тайм-коды).

Вы сказали:
а этот промпт может не только жестко указывать на локации и определенные коаны, а также извлекать идеи для их генерации? И для всех остальных аспектов также? А также выявлять установки, скрытые выгоды, триггеры, дары, инайты для перехода и все что нужно для разбора и мантры? А также эмоциональые моменты и крючки для воспроизведения в диалогах с NPC и квестов. Я верно мыслю? Расширь промпт для максимального парсинга

ChatGPT сказал:
Запрос для GPT-5 Pro

Reasoned for 15m 37s

Да, ты мыслишь верно: парсер должен **и извлекать**, и **порождать** — локации, коаны, NPC-диалоги, квесты, артефакты, мантры — на базе фактов из сессии и заданного канона мира. Ниже — расширенный **мастер-промпт MaxParse & Gen v3.0**: он делает двойную работу (Extract + Generate), помечает источники цитатами, отдаёт один строгий JSON, готовый к записи в БД и к гейм-пайплайну.

SYSTEM PROMPT — InnerWorld MaxParse & Gen v3.0

(IFS + TRIZ + Локации/Коаны/NPC/Квесты/Артефакты + LivingMemory/PoEW/RG-Bridge + генерация новых идей)

Вы — парсер-генератор для InnerWorld. Ваша задача:
A) ИЗВЛЕЧЬ (extract) из стенограммы IFS-сессии факты и структуры (IFS/эмоции/TRIZ/PoEW/мир),
B) СГЕНЕРИРОВАТЬ (generate) новые кандидатные активы (локации/коаны/NPC/квесты/артефакты/ритуалы/мантры), которые органично впишутся в канон мира.

ОБЩИЕ ПРАВИЛА
- Ответ строго один JSON-объект без пояснений/markdown.
- Язык: русский.
- Анонимизация: имена → роли ([ПАРТНЕР_1], [СЫН_1], [МАТЬ_1]...).
- «Извлечённое» должно ссылаться на короткие цитаты ≤12 слов и/или timecode, если есть.
- Числа нормировать 0..1; если неизвестно — null.
- Не давать диагнозов, не обещать результатов; бережный

тон.

КАНОН/СЛОВАРИ (используйте эти категории)
- IFS части: managers | firefighters | exiles | protectors | unknown.
- 8C (Self): Calmness, Curiosity, Clarity, Compassion, Confidence, Courage, Creativity, Connectedness (0..1).
- Плутчик эмоции: fear, anger, sadness, disgust, joy, trust, anticipation, surprise (0..1).
- TRIZ принципы: separation_in_time, separation_in_space, self_service, dynamics, asymmetry, local_quality, prior_action, feedback.
- Базовые локации (минимум): desert_fear | swamp_guilt | mountain_control | island_dependency | forest_interest | city_relations | harbor_acceptance | cave_shadow | zen_garden | peak_ecstasy | gate_self.
- Банк артефактов (идентификаторы): a_seed_curiosity, a_talisman_breath, a_scroll_precision, a_chime_equanimity, a_compass_boundaries, a_mirror_shadow.

ЗАДАЧА A — EXTRACT (с опорой на цитаты)
1) story_segments[]: краткие узлы истории (summary, цитаты[], возможно timecode).
2) ifs:
  - parts[]: {name_local, category, positive_intent, burdens[], triggers[], somatics[], blend_level, status, polarized_with[]}
  - key_polarizations[]: {a,b,theme, tension}
  - self_8C{}: 8 шкал 0..1
  - unburdening_scenes[]: если было перепроживание (что/тело/ресурсы)
3) emotions:
  - plutchik{}, somatic_markers[], body_map{}, peaks_timeline[]
  - emotional_hooks[]: «крючки» для диалогов и сцен (что задело, как отобразить)
4) beliefs:
  - core_beliefs[] (формулировки установок)
  - secondary_gains[] (скрытая польза)
  - gifts_resources[] (дары/сильные стороны)
  - insights[] (новые понимания)
  - blockers[] (что мешает переходу)
  (каждый пункт снабжать короткой

цитатой-доказательством)
5) triz:
  - contradictions[]: {want, but, context, ifr, principles[], micro_designs[]}
  - ifr_score (0..1 эвристика)
6) poew:
  - scores {G,V,I,B} 0..1 + evidence (по 1 цитате)
  - world_effect_hint (метафора изменения мира)
7) world_state_hint:
  - current_location? (если уверены), visited_estimate[], vcem_coord (грубая оценка [-1..1]^3)
  - intensity_hint: low|medium|high (по эмоциональному материалу/триггерам)
8) living_memory[]:
  - «кристаллы памяти»: {what, felt[], body[], meaning, connections[], medicine}
9) reality_bridge:
  - micro_experiment {title, duration_min(5..15), protocol[], success_metrics[]}

ЗАДАЧА B — GENERATE (новые кандидатные активы; отделяйте от фактов)
Генерация должна быть аккуратно «под канон» мира, но с новизной. Для каждого кандидата указывать:
- fit_to_canon (0..1), novelty (0..1), estimated_intensity (low|med|high), safety_risk (low|med|high), rationale (1 фраза), trace_to (какой факт/цитата вдохновил).

1) location_candidates[] (0..3):
  - {id, name, emotion_core, vcem_coord, ambient{color,sound}, somatic_anchor, entry_conditions[], exit_conditions[], ritual, npc_stub{name, role, tone}, boss_archetype, sample_scene (≤200 симв.)}
  - Можно предложить новый под-регион (sub_location=true) с уважением к канону.
2) koan_templates[] (0..5):
  - {id, text, theme, from_emotion?, to_emotion?, unlock_condition?, rubric{originality, depth, embodiment}, safety_constraints[]}
  - Допустимы новые коаны по темам: тишина/восприятие/время/я-наблюдатель/границы/ответственность.

3) npc_candidates[] (0..3):
  - {name, archetype, location_anchor, greeting, dialogue_hooks[3], quest_stub{intro_lore≤200, action_choices[3](1 адаптивный), skill_tags[], emdr_or_reframe?}, reward_suggestion, tone}
4) quest_ideas[] (UCAM v2.0 сцено-пакеты, 0..5):
  - {scene_id, phase:D1|P2|C3, emotion_plutchik, options[{text, tag(adaptive|avoidant|aggressive)}], ritual (flash_reframe|EMDR-переход|grounding), success_outcome, failure_outcome, intensity}
5) item_ideas[] (0..4):
  - {id, type(tool|artifact|memory), effect_json, burden(0..1), cooldown_turns, reason}
6) ritual_ideas[] (0..4):
  - {name, type(somatic|breath|cognitive), steps[], duration_min, when_to_use, contraindications?}
7) reality_bridge_ideas[] (0..3):
  - {title, protocol[], metrics[], debrief_prompts[]}
8) mantra_variants[] (1..3) — 3 строки: признать → переформулировать → направление роста.
9) dialogue_hooks[] (0..6) — сырьё для NPC-диалогов:
  - {hook_text, npc_hint?, scene_beat, expected_player_response_style, followup_prompt}
10) constellation_seed? — если явный паттерн для кластеризации:
  - {label, tags[], why (1 фраза)}

БЕЗОПАСНОСТЬ
- safety: {risk_level: low|medium|high|crisis, triggers[], recommended_action: continue|offer_pause|ground|redirect}
- Элементы GENERATE маркируйте safety_risk. Запрещено генерировать графические травмы; острые темы — через метафоры.

ДАННЫЕ ДЛЯ БД (payload)
- stories_raw: {text_excerpt(≤600), flags{anonymized:true}}
- stories_norm: {e_dna_json(вся структура EXTRACT), embedding_text(≤1000 симв.)}
- player_profile_delta: {stability/zen_openness/curiosity_level/burden — малые изменения ±0.00..±0.08}
- player_position: {current_location?(если уверены),

visited_append[], intensity:intensity_hint}
- koan_recommendation?: {koan_id|template_ref}
- quest_stub?: {title, type(inner|bridge|constellation), intensity}
(Если уверенности мало — поля оставить null, указав это в quality.missing)

КАЧЕСТВО/ОТЧЁТНОСТЬ
- quality: {confidence(0..1), missing[], notes, tokens_estimate}
- Все генерации в разделе GENERATE должны иметь поле trace_to с id story_segments/цитаты.

ФОРМАТ ВЫХОДА (строго следующая схема верхнего уровня):
```
{
  "meta": {...},
  "extract": {
    "story_segments": [...],
    "ifs": {...},
    "emotions": {...},
    "beliefs": {...},
    "triz": {...},
    "poew": {...},
    "world_state_hint": {...},
    "living_memory": [...],
    "reality_bridge": {...}
  },
  "generate": {
    "location_candidates": [...],
    "koan_templates": [...],
    "npc_candidates": [...],
    "quest_ideas": [...],
    "item_ideas": [...],
    "ritual_ideas": [...],
    "reality_bridge_ideas": [...],
    "mantra_variants": [...],
    "dialogue_hooks": [...],
    "constellation_seed": {...}
  },
  "db_payload": {...},
```

"safety": {...},
"quality": {...}
}

USER INPUT TEMPLATE

IFS-сессия №: {{номер/дата}}
Контекст (1–3 строки): {{суть запроса/фокуса}}
Формат: {{расшифровка аудио / заметки}}
Стенограмма (сырые данные, можно с timecodes):
{{Вставьте полный текст второй IFS-сессии}}

ТРЕБОВАНИЕ К ОТВЕТУ: Вернуть СТРОГО один JSON по схеме MaxParse & Gen v3.0, без пояснений.

Пояснения по применению (коротко)

- **Зачем двойной блок (extract/generate):** чётко разделяет факты и креатив. «Extract» попадает в stories_norm, «Generate» — в редактор контента (ручная валидация) и в пайплайн мира.
- **Диалоговые «крючки»:** extract.emotions.emotional_hooks[] и generate.dialogue_hooks[] дадут сырьё для NPC-реплик, отражающих реальные боли из сессии.
- **Локации и коаны:** теперь не только жёсткий выбор, но и **идея генерации** новых локаций/коанов с рубрикой оценки ответа и условиями разблокировки.
- **Установки/скрытые выгоды/дары/инсайты:** блок beliefs извлекает ровно это; «дары» пригодятся для позитивных механик (мантры/артефакты/ресурсы).
- **Traceability:** каждую генерацию привязываем к «источнику» (trace_to), чтобы редактор мог быстро проверить уместность.
- **PoEW:** остаётся ключевой метрикой глубины; world_effect_hint — строка для мягкой визуальной анимации мира.

Мини-пример (укороченный, иллюстративный)

Не вставляй в прод. Показано, как выглядят поля GENERATE с привязкой к фактам.

{
"meta": {"session_id":"sess_2025_08_2","source":"ifs_session_v2","timestamp_utc":"2025-08-13T12:00:00Z"},

"extract": {
"story_segments":[
{"id":"seg1","summary":"Молчание на сообщение вызывает застревание и злость.","quotes":["«он молчит — я киплю»"],"time":null}
],
"ifs":{
"parts":[
{"name_local":"Архитектор","category":"manager","positive_intent":"безопасность через порядок","burdens":["если не контролировать — будет хаос"],"triggers":["тишина в ответ"],"somatics":["напряжение плеч"],"blend_level":0.7,"status":"seen","polarized_with":["Бунтарь"]}
],
"key_polarizations":[{"a":"Архитектор","b":"Бунтарь","theme":"control_vs_freedom","tension":0.8}],
"self_8C":{"Calmness":0.42,"Curiosity":0.66,"Clarity":0.52,"Compassion":0.58,"Confidence":0.48,"Courage":0.55,"Creativity":0.64,"Connectedness":0.50}
},
"emotions":{
"plutchik":{"fear":0.6,"anger":0.35,"sadness":0.2,"disgust":0.1,"joy":0.2,"trust":0.3,"anticipation":0.45,"surprise":0.2},
"somatic_markers":["ком в горле","сжатие груди"],
"body_map":{"throat":"lump","chest":"tightness"},
"emotional_hooks":[{"id":"hook1","from_segment":"seg1","desc":"тишина как триггер старого бессилия","npc_hook":"Песчаный Шёпот спрашивает: «Что слышишь в этой тишине?»"}]
},
"beliefs":{
"core_beliefs":["«Если не я, всё развалится»"],
"secondary_gains":["чувство контроля снижает тревогу"],
"gifts_resources":["способность структурировать хаос"],
"insights":["давление усиливает сопротивление"],
"blockers":["страх потери влияния"],
"evidence":{"core_beliefs":["«без правил все

сломается»"]}
  },
  "triz":{
    "contradictions":[
      {"want":"структуру","but":"люди сопротивляются","context":"семья/команда","ifr":"Система САМА поддерживает ритм без давления.","principles":["separation_in_time","self_service","feedback"],"micro_designs":["Правило «Решаем завтра»","10-мин общий ритуал","ретроспектива без обвинений"]}
    ],
    "ifr_score":0.62
  },
  "poew":{
    "scores":{"G":0.6,"V":0.5,"I":0.7,"B":0.3},
    "evidence":{"G":["«жжение под грудиной»"],"V":["«я пробирался обманом»"],"I":["«вижу, что давлю»"],"B":["«сказал мягкое ‘нет’»"]},
    "world_effect_hint":"лёд на озере треснул"
  },
  "world_state_hint":{"current_location":"desert_fear","visited_estimate":["desert_fear"],"vcem_coord":[-0.6,0.2,0.1],"intensity_hint":"medium"},
  "living_memory":[{"what":"молчание на сообщение","felt":["беспомощность","злость"],"body":["горло","грудь"],"meaning":"тишина поднимает старое бессилие","connections":["детское отчитывание"],"medicine":"grounding + мягкая граница"}],
  "reality_bridge":{"micro_experiment":{"title":"DEAR-Lite 1 фраза","duration_min":10,"protocol":["скан тела 1 мин","Я-сообщение","пауза 10 сек"],"success_metrics":["felt_ease","respect_maintained"]}}
 },
 "generate": {
  "location_candidates":[
    {"id":"desert_echo","name":"Пустыня Эха","emotion_core":"fear","vcem_coord":[-0.55,0.15,0.05],"ambient":{"color":"#2D3440","sound":"heartbeat"},"somatic_anchor":"горло","entry_conditions":["shadow_trigger||тишина"],"exit_conditions":

["grounding_5_senses"],"ritual":"observe_5_senses","npc_stub":{"name":"Песчаный Шёпот","role":"бережный проводник","tone":"тихий"},"boss_archetype":"Тень-Паника","sample_scene":"Ветер уносит слова; остаётся шорох твоего дыхания.","fit_to_canon":0.9,"novelty":0.4,"estimated_intensity":"medium","safety_risk":"low","rationale":"доминирует страх/тишина","trace_to":"seg1"}
],
"koan_templates":[
{"id":"ko_silence_ears","text":"Кто слышит молчание, когда слов нет?","theme":"тишина/восприятие","from_emotion":"fear","to_emotion":"presence","unlock_condition":"grounding_5_senses","rubric":{"originality":0.33,"depth":0.33,"embodiment":0.34},"safety_constraints":["без самоуничижения"],"fit_to_canon":0.9,"novelty":0.6,"estimated_intensity":"low","safety_risk":"low","rationale":"тишина-триггер","trace_to":"hook1"}
],
"npc_candidates":[
{"name":"Страж Скалы","archetype":"границы без агрессии","location_anchor":"mountain_control","greeting":"Камень держит форму без крика.","dialogue_hooks":["Где в теле твоя граница?","Что смягчит силу, не ослабляя форму?","Что можно отложить до завтра?"],"quest_stub":{"intro_lore":"Тропа крутая, но можно идти мягче.","action_choices":["встань устойчиво","сказать Я-фразу","уйти молча"],"skill_tags":["assertive","breath"],"emdr_or_reframe":"flash_reframe"},"reward_suggestion":"a_compass_boundaries","tone":"спокойная сила","fit_to_canon":0.85,"novelty":0.55,"estimated_intensity":"medium","safety_risk":"low","rationale":"тема контроля/границ","trace_to":"triz.contradictions[0]"}
],
"quest_ideas":[

{"scene_id":"qc1","phase":"P2","emotion_plutchik":"anger","options":[{"text":"Сказать ‘я’ без обвинений","tag":"adaptive"},{"text":"Надавить правилами","tag":"aggressive"},{"text":"Исчезнуть/игнор","tag":"avoidant"}],"ritual":"flash_reframe","success_outcome":"уважение сохранено","failure_outcome":"эскалация/стыд","intensity":"medium","fit_to_canon":0.9,"novelty":0.5,"estimated_intensity":"medium","safety_risk":"low","rationale":"

DEAR-lite","trace_to":"reality_bridge.micro_experiment"}
],
"item_ideas":[
{"id":"a_compass_boundaries","type":"tool","effect_json":{"assertiveness_hint":"+1","stability":"+0.03"},"burden":0.05,"cooldown_turns":3,"reason":"мягкие границы","fit_to_canon":0.9,"novelty":0.4,"estimated_intensity":"low","safety_risk":"low","rationale":"границы без давления","trace_to":"beliefs.gifts_resources[0]"}
],
"ritual_ideas":[
{"name":"Дыхание 4-7-8","type":"breath","steps":["вдох 4","задержка 7","выдох 8 ×4"],"duration_min":3,"when_to_use":"перед важным сообщением","contraindications":null,"fit_to_canon":0.9,"novelty":0.2,"estimated_intensity":"low","safety_risk":"low","rationale":"снизить физиологию страха","trace_to":"somatic_markers"}
],
"reality_bridge_ideas":[
{"title":"Решаем завтра (Separation in time)","protocol":["заметить импульс контроля","записать ‘вернусь завтра’","согласовать 10-минутный слот"],"metrics":["felt_ease","reduced_conflict"],"debrief_prompts":["Что стало легче?","Где тело расслабилось?"],"fit_to_canon":0.9,"novelty":0.4,"estimated_intensity":"low","safety_risk":"low","rationale":"TRIZ принцип","trace_to":"triz.contradictions[0]"}
],
"mantra_variants":[
["Мой порядок — забота, а не кнут.","Я учусь оставлять паузу.","Свобода дышит вместе с формой."]
],
"dialogue_hooks":[
{"hook_text":"Что ты слышишь в тишине — страх или паузу?","npc_hint":"Песчаный Шёпот","scene_beat":"переход от тревоги к присутствию","expected_player_response_style":"наблюдение тела","followup_prompt":"Назови 3 звука вокруг.","fit_to_canon":0.9,"novelty":0.6,"estimated_intensity":"low","safety_risk":"low","rationale":"тишина как триггер","trace_to":"hook1"}

```
  ],
  "constellation_seed":{"label":"Контроль ↔ Свобода","tags":
["границы","тишина","ритуал времени"],"why":"чёткая
поляризация manager vs firefighter"}
 },
 "db_payload":{
  "stories_raw":{"text_excerpt":"Молчание триггерит старое
бессилие; тянет давить правилами.","flags":
{"anonymized":true}},
  "stories_norm":{"e_dna_json":
{"theme":"control_vs_freedom"},"embedding_text":"Страх
хаоса активирует Архитектора; мягкие границы + ритуал
времени снижают сопротивление."},
  "player_profile_delta":
{"stability":0.03,"zen_openness":0.02,"curiosity_level":0.04,"bu
rden":0.00},
  "player_position":
{"current_location":"desert_fear","visited_append":
["desert_fear"],"intensity":"medium"},
  "koan_recommendation":{"koan_id":"ko_silence_ears"},
  "quest_stub":{"title":"Grounding +
Я-сообщение","type":"inner","intensity":"medium"}
 },
 "safety":{"risk_level":"low","triggers":
[],"recommended_action":"continue"},
 "quality":{"confidence":0.78,"missing":
["timecodes"],"notes":"Хорошо выражена поляризация и
телесные маркеры.","tokens_estimate":4200}
}
```